

Universidad Técnica Federico Santa María
Departamento de Matemáticas

Transformers para la predicción de series de tiempo no equiespaciadas

Memoria para optar al título de
Ingeniero Civil Matemático
(Ingeniería Civil Matemática)

Presentada por
Gerald Méndez

Profesor guía: Carlos Valle
Profesor co-guía: Francisco Cuevas

Valparaíso, 2025

La familia va mucho más allá de lo que socialmente consideramos como tal. La sangre no tiene porqué significar un vínculo.

Agradezco a mi familia.

Al conocimiento, que sea de todos y para todos.

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Problema de investigación	2
1.3. Hipótesis y preguntas de investigación	2
1.4. Objetivos	3
1.4.1. Objetivo general	3
1.4.2. Objetivos específicos	3
1.5. Enfoque y resumen de la propuesta	4
1.6. Alcances y limitaciones	5
1.7. Estructura de la memoria	6
2. Estado del arte	7
2.1. Modelos clásicos para series de tiempo irregulares	7
2.1.1. Regularización e imputación sobre una malla equiespaciada	7
2.1.2. Modelos en tiempo continuo: CARMA y modelos de espacio de estados	8
2.1.3. Procesos gaussianos para series irregulares	8
2.2. Aprendizaje profundo para series de tiempo irregulares	9
2.2.1. Redes recurrentes con manejo explícito de tiempo y <i>missingness</i>	9
2.2.2. Modelos en tiempo continuo: Neural ODE y variantes	9
2.2.3. Ecuaciones diferenciales controladas y Neural CDE	10
2.3. Transformers para series de tiempo	11
2.3.1. Transformers para series equiespaciadas	11
2.3.2. Positional encodings continuos y representaciones del tiempo	11
2.3.3. Transformers sobre observaciones irregulares y modelos híbridos	12
2.3.4. Agregación por <i>patches</i> y baselines simples para series irregulares	13
2.4. Clasificación de series de tiempo irregulares	14
2.4.1. Métodos probabilísticos y kernels para series irregulares	14
2.4.2. Modelos de deep learning para clasificación irregular	14
2.4.3. Transformers para clasificación de series irregulares	15
2.5. Discusión y brechas de investigación	15
3. Series de tiempo y formulación del problema	17
3.1. Conceptos básicos de series de tiempo	17
3.1.1. Definición y notación	17
3.1.2. Series equiespaciadas vs. no equiespaciadas	18
3.2. Datos considerados en la memoria	19
3.3. Problema de predicción en series de tiempo	19
3.3.1. Predicción a partir de una ventana de historia	19

3.3.2.	Horizontes de predicción y offsets	20
3.4.	Construcción de ejemplos supervisados	20
3.4.1.	Ventanas deslizantes y tamaño de la historia	20
3.4.2.	Definición de la entrada y la salida de cada ejemplo	21
3.5.	Formulación como problema de aprendizaje supervisado	22
3.5.1.	Función objetivo	22
3.5.2.	Entrenamiento, validación y prueba	22
3.6.	Resumen del capítulo	22
4.	Transformers para series de tiempo no equiespaciadas	24
4.1.	Recordatorio de la arquitectura Transformer	24
4.2.	Transformers en series de tiempo	25
4.2.1.	Ventajas	25
4.2.2.	Desafíos con timestamps irregulares	25
4.3.	Principios de adaptación para series no equiespaciadas	26
4.4.	Del marco conceptual al modelo propuesto	26
4.5.	Resumen del capítulo	27
5.	CT-Transformer para series de tiempo no equiespaciadas	28
5.1.	Objetivos de diseño	29
5.2.	Representación de entradas y construcción de ejemplos	29
5.2.1.	Separación entre valores y timestamps	29
5.2.2.	Ventanas de historia y targets	30
5.2.3.	Construcción de secuencias con token objetivo	31
5.3.	Arquitectura de CT-Transformer	32
5.3.1.	Embedding de valores continuos	32
5.3.2.	Encoding temporal continuo	32
5.3.3.	Embedding de flag historia/target	35
5.3.4.	Combinación de embeddings y paso por el encoder	35
5.3.5.	Cabeza de regresión en el token objetivo	38
5.3.6.	Variante con resumen atencional de la historia	39
5.3.7.	Variante encoder–decoder	39
5.4.	Manejo de historias variables y offsets de predicción	42
5.5.	Función de pérdidas, métricas y bucle de entrenamiento	42
5.5.1.	Función de pérdida	42
5.5.2.	Métricas de evaluación	43
5.5.3.	Optimización y scheduler	43
5.6.	Inferencia y pronósticos multi-step	43
5.7.	Comentarios sobre complejidad y limitaciones	44
6.	Experimentos	45
6.1.	Objetivos experimentales	45
6.2.	Escenario experimental	46
6.2.1.	Variables, partición temporal y preprocesamiento	46
6.3.	Modelos comparados	46
6.3.1.	Familia CT-Transformer	46
6.3.2.	Variantes encoder–decoder	47
6.3.3.	Baselines del estado del arte	47
6.3.4.	Baselines simples	48

6.3.5. Ablaciones	48
6.4. Configuración de entrenamiento	48
6.5. Métricas de evaluación	48
6.6. Agregación y análisis estadístico	49
6.7. Costo computacional	49
6.8. Reproducibilidad	49
6.9. Resumen metodológico	49
7. Resultados y discusión	50
7.1. Cobertura final del benchmark	50
7.2. Metodología de presentación y análisis de resultados	50
7.3. Resultado principal	51
7.4. Comparación contra baselines	52
7.5. Variantes encoder-decoder	52
7.6. Variantes CT-Transformer	53
7.7. Ablaciones	54
7.8. Costo computacional	54
7.9. Significancia estadística y alcance	54
7.10. Discusión sobre MAPE	55
7.11. Síntesis del capítulo	55
8. Conclusiones y trabajo futuro	57
8.1. Conclusiones principales	57
8.2. Contraste con la hipótesis	58
8.3. Limitaciones	59
8.4. Trabajo futuro	59
8.5. Cierre	59
A. Fundamentos de redes neuronales y optimización	63
A.1. Introducción	63
A.2. Neurona artificial	63
A.2.1. Modelo básico	63
A.2.2. Interpretación geométrica	64
A.3. Redes neuronales feedforward	65
A.3.1. Capas densas y notación matricial	65
A.3.2. Composición de capas	65
A.4. Función de pérdida	66
A.4.1. Pérdida para problemas de regresión	66
A.4.2. Regularización en la función de pérdida	67
A.5. Descenso de gradiente y retropropagación	67
A.5.1. Descenso de gradiente	67
A.5.2. Retropropagación (<i>backpropagation</i>)	67
A.6. Funciones de activación	68
A.6.1. Requisitos básicos	68
A.6.2. Sigmoide y tangente hiperbólica	68
A.6.3. ReLU y variantes	69
A.7. Problemas en el entrenamiento de redes profundas	69
A.7.1. Gradientes que se desvanecen y explotan	69
A.7.2. Regularización y generalización	70

A.8. Optimizadores avanzados: Adam	70
A.8.1. Motivación	70
A.8.2. Algoritmo Adam	70
A.9. Relación con CT-Transformer	71
A.10. Resumen del anexo	72
B. Detalles de optimización y scheduler	73
B.1. AdamW	73
B.2. Calentamiento lineal y decaimiento coseno	73
B.3. Diferencia con CosineAnnealingLR	74

Índice de figuras

5.1.	Comparación entre el encoding sinusoidal tradicional y el encoding temporal continuo. Ambos paneles cubren el mismo rango de tiempo relativo y usan la misma familia de funciones seno y coseno, la diferencia está en la coordenada de entrada: una grilla discreta de posiciones en el caso tradicional y una grilla temporal más densa, con valores intermedios, en el caso continuo.	33
5.2.	Comparación entre un encoding sinusoidal fijo y el encoding Time2Vec aprendido por el mejor modelo Time2Vec disponible en el archivo actual de resultados: <code>Custom-Time2VecBias-Small</code> , dataset 4, semilla 126, con MSE de test 0,2094. Ambos paneles usan las mismas coordenadas temporales intermedias y la misma escala de color.	34
5.3.	Esquema del encoder utilizado en la arquitectura base. Las representaciones iniciales de historia y tokens objetivo se apilan en la matriz $H^{(0)}$ y luego atraviesan N bloques encoder compuestos por normalización, auto-atención multi-cabeza, conexión residual, normalización, red feedforward y una segunda conexión residual.	36
5.4.	Esquema de la variante encoder-decoder. La historia observada se procesa en el encoder para producir una memoria contextual, mientras que los tokens objetivo se procesan en el decoder mediante auto-atención multi-cabeza y atención cruzada multi-cabeza hacia la salida del encoder.	41
A.1.	Ejemplo bidimensional de un hiperplano que separa dos clases linealmente separables: las cruces negras pertenecen a una clase y los círculos rojos a la otra.	64
A.2.	Esquema de una red neuronal <i>feedforward</i> con dos capas ocultas: los vectores de entrada $x_1, \dots, x_{\text{in}}$ se transforman mediante las matrices de pesos W_1 y W_2 en representaciones internas, para finalmente producir las salidas $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{\text{out}}$	66

Índice de tablas

7.1. Cobertura final usada en el análisis.	50
7.2. Resultados finales de modelos representativos.	52
7.3. Resultados finales de variantes encoder–decoder optimizadas.	53
7.4. Resultados finales de variantes encoder de CT-Transformer.	53
7.5. Ablaciones finales respecto de CT-Transformer base.	54

Capítulo 1

Introducción

1.1. Contexto y motivación

La predicción de series de tiempo es un problema central en múltiples áreas de la ingeniería y las ciencias aplicadas, tales como la minería, la energía, las telecomunicaciones, la meteorología o la salud. En estos dominios es habitual contar con grandes volúmenes de datos temporales recolectados por sensores, sistemas de monitoreo o plataformas transaccionales. Poder anticipar el comportamiento futuro de estas series permite, entre otras cosas, planificar operaciones, detectar anomalías, optimizar recursos y reducir costos asociados a fallas o ineficiencias.

Tradicionalmente, la modelación de series de tiempo se ha abordado mediante modelos estadísticos como ARIMA, modelos autorregresivos vectoriales (VAR) o modelos de suavizamiento exponencial. Más recientemente, el desarrollo de técnicas de *aprendizaje profundo* ha dado lugar a arquitecturas como las redes recurrentes (RNN), las redes LSTM y GRU, así como modelos convolucionales específicamente adaptados a datos temporales. Sin embargo, en los últimos años ha emergido con fuerza una familia de modelos basada en el mecanismo de atención, conocida como *Transformers*, que ha demostrado un rendimiento sobresaliente en tareas de procesamiento de lenguaje natural y visión computacional.

La mayoría de las aplicaciones exitosas de Transformers a series de tiempo asume implícita o explícitamente que las observaciones están *equiespaciadas* en el tiempo, es decir, que el intervalo entre mediciones es constante. Esta suposición simplifica la definición de posiciones en la secuencia y permite reutilizar sin cambios el *positional encoding* sinusoidal clásico. No obstante, en muchos sistemas reales los datos no se registran con una frecuencia perfectamente regular: pueden existir lapsos de tiempo sin mediciones, lecturas adicionales cuando se detectan eventos relevantes, fallas en los sensores o cambios en la tasa de muestreo. En estas situaciones se habla de *series de tiempo no equiespaciadas*.

Trabajar con series no equiespaciadas introduce desafíos adicionales. Una práctica habitual consiste en *regularizar* la serie mediante técnicas como el re-muestreo (por ejemplo, cada 5 minutos) e imputación de valores faltantes. Si bien esto permite reutilizar modelos diseñados para series equiespaciadas, también introduce sesgos y pérdida de información temporal, especialmente cuando la irregularidad es alta o cuando la dinámica del sistema depende críticamente de la distribución real de los tiempos de observación.

En este contexto, surge la motivación de investigar si es posible adaptar la arquitectura Transformer para trabajar de manera más directa con timestamps continuos, evitando imponer una grilla temporal artificial y aprovechando explícitamente la información sobre la separación temporal entre mediciones.

1.2. Problema de investigación

El problema que se aborda en esta memoria puede resumirse de la siguiente forma:

Desarrollar una estrategia para utilizar modelos tipo Transformer en la predicción de series de tiempo no equiespaciadas, incorporando los timestamps de forma explícita mediante una representación temporal continua, sin imponer una grilla temporal fija, y comparar su desempeño con baselines simples y modelos adaptados a observaciones irregulares.

En particular, se busca responder si un Transformer diseñado para operar con tiempos reales (por ejemplo, segundos desde un origen de referencia), y no sólo con índices enteros de posición, es capaz de:

- Capturar adecuadamente la dinámica de series de tiempo donde los intervalos entre mediciones son variables.
- Alcanzar un desempeño predictivo competitivo frente a baselines simples y modelos diseñados o adaptados para observaciones irregulares.
- Ofrecer una formulación lo suficientemente flexible como para manejar ventanas de historia de longitud variable y distintos horizontes de predicción, sin necesidad de rediseñar la arquitectura.

Este problema combina aspectos conceptuales—cómo representar el tiempo de forma continua en un Transformer—con aspectos prácticos de implementación y experimentación—cómo construir datasets, definir ventanas temporales y diseñar experimentos reproducibles para comparar distintos enfoques.

1.3. Hipótesis y preguntas de investigación

A partir del problema descrito, se plantea la siguiente hipótesis de trabajo:

Hipótesis de trabajo: *Una arquitectura tipo Transformer que represente los timestamps como variables continuas y distinga explícitamente entre tokens de historia y tokens objetivo puede aprender directamente de series de tiempo no equiespaciadas, sin regularizarlas previamente sobre una grilla temporal fija. Bajo esta formulación, el modelo puede alcanzar un desempeño predictivo competitivo frente a baselines simples y modelos adaptados a observaciones irregulares.*

De esta hipótesis se desprenden varias preguntas específicas:

- ¿Cómo representar los timestamps reales dentro de un Transformer sin convertir previamente la serie a una grilla equiespaciada?

- ¿Es posible diseñar un esquema de construcción de ventanas de historia y horizontes de predicción que permita al modelo entrenarse con longitudes de contexto variables y distintos offsets de predicción?
- ¿Qué aporte muestran los componentes principales de la propuesta, en particular el encoding temporal continuo y el identificador historia–objetivo?
- ¿Cómo se compara la arquitectura propuesta, en error predictivo y costo computacional, frente a baselines simples y modelos adaptados a observaciones irregulares?

Responder estas preguntas requiere tanto de una formulación conceptual clara de la arquitectura propuesta como de una campaña experimental controlada, que permita aislar el efecto de la representación temporal respecto de otros factores.

1.4. Objetivos

1.4.1. Objetivo general

El objetivo general de esta memoria es:

Diseñar, implementar y evaluar la familia de arquitecturas propuesta para la predicción de series de tiempo no equiespaciadas, incorporando explícitamente los timestamps mediante una representación temporal continua, y analizar su desempeño, costo computacional y limitaciones frente a baselines simples y modelos adaptados a observaciones irregulares.

1.4.2. Objetivos específicos

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. **Revisar el estado del arte** en modelos de series de tiempo, con énfasis en arquitecturas de aprendizaje profundo (RNN, CNN, Transformers) y en propuestas que abordan explícitamente el problema de tiempos de muestreo irregulares.
2. **Formalizar el problema** de predicción de series de tiempo no equiespaciadas, definiendo la notación, las variables involucradas y los escenarios de predicción a considerar (un paso adelante, múltiples pasos, distintos horizontes, etc.).
3. **Diseñar una arquitectura Transformer** capaz de:
 - Proyectar features continuas multivariadas a un espacio latente común.
 - Incorporar un encoding temporal continuo basado en timestamps reales y una escala de tiempo configurable.
 - Diferenciar explícitamente entre tokens de historia y tokens objetivo mediante embeddings adicionales.
4. **Implementar un pipeline completo** de entrenamiento e inferencia que incluya:
 - Construcción de ventanas de historia y targets para series no equiespaciadas.

- Entrenamiento del modelo con historias de longitud fija o variable, y con uno o varios offsets de predicción¹ posibles, lo que posibilita estudiar el efecto del horizonte de predicción sobre el desempeño.
 - Módulos de escalado, funciones de pérdida, métricas y bucles de entrenamiento reproducibles.
5. **Diseñar y ejecutar experimentos** sobre datos reales procesados que permitan comparar:
- La familia de arquitecturas propuesta con encoding temporal continuo.
 - Baselines simples, ablaciones y modelos del estado del arte adaptados a observaciones irregulares.

utilizando métricas de error de regresión (MSE, RMSE, MAE, MAPE), análisis de costo computacional y comparaciones emparejadas cuando exista cobertura experimental suficiente.

6. **Analizar los resultados** para contrastar la hipótesis de trabajo, evaluar la viabilidad del enfoque, identificar sus ventajas y limitaciones, distinguir evidencia preliminar de conclusiones definitivas, y proponer posibles líneas de trabajo futuro.

1.5. Enfoque y resumen de la propuesta

Para abordar los objetivos anteriores, en esta memoria se desarrolla la familia CT-Transformer (*Continuous Transformer*), adaptada específicamente a series de tiempo no equiespaciadas. De forma muy resumida, la propuesta se estructura en los siguientes componentes:

- **Módulo de embedding de valores** que proyecta las variables de entrada (por ejemplo, mediciones de sensores) desde un espacio de dimensión d_{in} a un espacio latente de dimensión d_{model} , común a todo el modelo.
- **Encoding temporal continuo** que recibe directamente los timestamps numéricos y construye vectores de dimensión d_{model} para cada observación, utilizando una normalización en base a una escala de tiempo configurable y un esquema sinusoidal continuo, con variantes aprendibles basadas en Time2Vec.
- **Identificador de rol historia–objetivo** que diferencia explícitamente entre tokens observados y tokens de predicción, permitiendo introducir uno o más tokens objetivo en la secuencia con sus timestamps correspondientes, pero sin conocer aún sus valores verdaderos.
- **Arquitectura base con encoder Transformer**, compuesta por múltiples bloques de atención multi-cabeza y capas feed-forward, que opera sobre la secuencia completa (historia + tokens objetivo), respetando opcionalmente una máscara causal.

¹En este contexto, un *offset de predicción* es la distancia entre el último punto de la historia usada como entrada y el instante futuro que se desea predecir. Por ejemplo, un offset de 1 puede corresponder al siguiente punto de la serie, mientras que un offset de 20 puede representar predecir veinte pasos más adelante.

- Variantes **encoder–decoder**, donde la historia se codifica por separado y los tokens objetivo se procesan en un decoder con atención cruzada hacia la memoria histórica.
- **Cabeza de regresión** que toma las representaciones latentes de los tokens objetivo y las proyecta al espacio de salida.

Sobre esta arquitectura se construye un conjunto de herramientas de datos y entrenamiento que permiten:

- Generar ventanas de historia y objetivos desde una serie temporal ordenada, respetando la naturaleza no equiespaciada de los timestamps.
- Entrenar el modelo con historias de longitud variable y múltiples offsets de predicción, lo que permite estudiar el efecto del horizonte sobre el desempeño.
- Evaluar el modelo mediante un benchmark reanudable con métricas de regresión estándar y registro de costo computacional.

Los fundamentos matemáticos de redes neuronales, funciones de activación, algoritmos de optimización y aspectos generales de entrenamiento de modelos de aprendizaje profundo se desarrollan en anexos independientes, con el fin de mantener el cuerpo principal de la memoria enfocado en la propuesta específica y sus resultados, sin perder rigurosidad en la presentación de los conceptos básicos.

1.6. Alcances y limitaciones

Este trabajo se concentra en el problema de *predicción puntual* de series de tiempo, es decir, en entregar un valor esperado para la variable de interés en uno o varios tiempos futuros. No se abordan explícitamente modelos probabilísticos completos (por ejemplo, distribución de probabilidad sobre futuros posibles) ni técnicas de *forecasting* probabilístico como cuantiles o intervalos de confianza, aunque se mencionan brevemente como posibles extensiones.

Asimismo, la evaluación está diseñada para cubrir 5 datasets reales procesados. El objetivo no es presentar un modelo universalmente óptimo para todas las series de tiempo no equiespaciadas, sino estudiar en detalle la viabilidad y el comportamiento de la arquitectura CT-Transformer en escenarios concretos.

Por último, si bien la implementación se realiza sobre la biblioteca PyTorch y se incluye un pipeline completo de entrenamiento e inferencia, aspectos de despliegue en producción, integración con sistemas de monitoreo en tiempo real o escalamiento distribuido quedan fuera del alcance de esta memoria y se mencionan únicamente como posibles líneas de trabajo futuro.

Todo el proyecto se deja alojado en un repositorio público de GitHub, con el fin de facilitar la reproducibilidad, la colaboración y la extensión por parte de otros investigadores interesados en el tema.

1.7. Estructura de la memoria

El resto de este documento se organiza de la siguiente manera:

- En el **Capítulo 2** se presenta el estado del arte en modelos de series de tiempo, revisando tanto enfoques clásicos como arquitecturas de aprendizaje profundo, con énfasis en Transformers y en técnicas para manejar series no equiespaciadas.
- En el **Capítulo 3** se formaliza el problema de predicción de series de tiempo no equiespaciadas, se introduce la notación utilizada y se describen los datasets empleados en los experimentos.
- En el **Capítulo 4** se revisan los componentes principales de la arquitectura Transformer aplicada a series de tiempo equiespaciadas, destacando el rol del positional encoding y las particularidades del uso de atención en este contexto.
- En el **Capítulo 5** se detalla la familia CT-Transformer, incluyendo el diseño del encoding temporal continuo, la construcción de secuencias con tokens objetivo, la arquitectura base con encoder y la variante encoder-decoder.
- En el **Capítulo 6** se describe la metodología experimental, los escenarios considerados y los procedimientos de entrenamiento y evaluación utilizados.
- En el **Capítulo 7** se presentan y analizan los resultados, comparando CT-Transformer con distintos baselines.
- Finalmente, en el **Capítulo 8** se resumen las conclusiones, se explicitan las limitaciones del estado actual y se proponen líneas de trabajo futuro.

Los anexos incluyen una exposición más detallada de los fundamentos de redes neuronales y algoritmos de optimización utilizados, así como información técnica adicional sobre la implementación y resultados complementarios de los experimentos.

Capítulo 2

Estado del arte

Este capítulo revisa los principales enfoques para modelar series de tiempo con marcas de tiempo irregulares, tanto para tareas de *forecasting* como de clasificación. El objetivo es situar la propuesta de esta memoria dentro del contexto de la literatura reciente, destacando las ventajas y limitaciones de cada familia de modelos, y las brechas que motivan el modelo basado en *Transformers* descrito en los capítulos siguientes.

2.1. Modelos clásicos para series de tiempo irregulares

2.1.1. Regularización e imputación sobre una malla equiespaciada

Una estrategia ampliamente utilizada para tratar series no equiespaciadas consiste en transformar el problema a un escenario equiespaciado. Típicamente se elige una malla temporal fija (por ejemplo, cada 5 o 15 minutos) y se aplica alguna combinación de:

- **Re-muestreo** (*resampling*) de las observaciones al grid objetivo.
- **Imputación** de valores faltantes mediante métodos simples (cero, promedio, *forward/backward fill*) o técnicas más avanzadas (interpolación, imputación múltiple, *splines*, *kalman smoothing*, etc.).

Sobre la serie regularizada se aplican luego modelos estándar como ARIMA, modelos de suavizamiento exponencial, redes recurrentes o *Transformers* para datos equiespaciados. Esta aproximación es sencilla de implementar y permite reutilizar modelos consolidados, sin embargo, presenta limitaciones relevantes:

- La elección del paso de malla introduce un *compromiso* entre resolución temporal y costo computacional.
- La imputación puede introducir *sesgos* sistemáticos cuando la irregularidad está asociada a la dinámica del sistema (por ejemplo, muestreo más denso durante eventos anómalos).
- Se pierde la posibilidad de formular horizontes de predicción directamente en tiempo continuo (por ejemplo, “15 minutos”, “1 hora”) de manera consistente con los *timestamps* originales.

Estas limitaciones motivan el desarrollo de modelos que trabajen directamente con tiempos continuos y sean capaces de manejar observaciones irregulares sin necesidad de re-muestreo agresivo.

2.1.2. Modelos en tiempo continuo: CARMA y modelos de espacio de estados

En el ámbito estadístico, una primera familia de modelos que trata de manera explícita el tiempo continuo son los procesos *CARMA* (*Continuous-time AutoRegressive Moving Average*) Brockwell, 2001a, 2001b. De forma análoga a los modelos ARMA discretos, los CARMA describen la evolución de un proceso estocástico mediante ecuaciones diferenciales lineales con ruido gaussiano, lo que permite modelar datos observados a tiempos irregulares sin re-muestreo explícito. La representación en espacio de estados facilita la inferencia mediante filtros y suavizadores de Kalman en tiempo continuo, y la evaluación de la verosimilitud para observaciones registradas en instantes arbitrarios.

Los modelos en espacio de estados continuos se han extendido a escenarios más generales (por ejemplo, procesos de difusión o modelos con parámetros dependientes del tiempo), pero siguen siendo mayoritariamente *lineales y gaussianos*. Esto limita su capacidad para capturar dinámicas altamente no lineales, frecuentes en aplicaciones industriales o biomédicas. Además, la extensibilidad a series multivariadas de alta dimensión suele implicar matrices de gran tamaño y costos de cómputo elevados.

2.1.3. Procesos gaussianos para series irregulares

Los *procesos gaussianos* (GP) constituyen otra herramienta probabilística natural para modelar series de tiempo irregulares, ya que la función de covarianza se evalúa directamente sobre los *timestamps* reales Williams y Rasmussen, 2006. Un GP define, para cualquier conjunto finito de instantes $\{t_1, \dots, t_T\}$, una distribución multivariada gaussiana

$$(y(t_1), \dots, y(t_T)) \sim \mathcal{N}(m, K),$$

donde $K_{ij} = k(t_i, t_j)$ está determinada por un *kernel* que codifica supuestos de suavidad, periodicidad o escala temporal.

Esta formulación tiene varias ventajas:

- Maneja *directamente* muestreo irregular: basta con evaluar el kernel en los tiempos observados.
- Proporciona *predicciones probabilísticas* (medias e intervalos de confianza) para cualquier tiempo futuro.
- Permite incorporar conocimiento de dominio mediante el diseño de kernels específicos (por ejemplo, kernels periódicos o *quasi-periodic*).

Sin embargo, la complejidad cúbica $O(T^3)$ en el número de observaciones limita el uso de GP exactos en series largas. Aunque existen aproximaciones escalables basadas en *inducing points*, estructuras de kernel especiales o submuestreo, el uso de GP puros en escenarios de alta frecuencia y larga duración sigue siendo desafiante, y la extensión a múltiples variables y tareas complejas (clasificación secuencial, detección de eventos) requiere diseños cuidadosos.

2.2. Aprendizaje profundo para series de tiempo irregulares

En la última década, los modelos de aprendizaje profundo han ganado protagonismo en el análisis de series de tiempo, tanto regulares como irregulares. A continuación se revisan tres líneas principales: (i) redes recurrentes con conocimiento explícito del tiempo, (ii) modelos en tiempo continuo basados en ecuaciones diferenciales, y (iii) modelos híbridos que combinan ideas de ambos mundos.

2.2.1. Redes recurrentes con manejo explícito de tiempo y *missingness*

Las *redes recurrentes* (RNN), incluyendo variantes como LSTM y GRU Cho et al., 2014; Chung et al., 2014; Gers et al., 2000; Hochreiter y Schmidhuber, 1997, son naturalmente adecuadas para procesar secuencias de longitud variable. Sin embargo, en su versión estándar asumen un paso de tiempo implícito (uno por paso de la RNN) y no modelan explícitamente los intervalos de tiempo entre observaciones. Para series irregulares, esto se ha abordado introduciendo *features* temporales y mecanismos de decaimiento aprendibles.

Un trabajo influyente en esta dirección es *GRU-D* Che et al., 2018, que extiende la GRU estándar incorporando dos ideas clave: (i) un vector de *máscaras* que indica qué variables están observadas o faltantes en cada instante, y (ii) un conjunto de *intervalos de tiempo* desde la última observación de cada variable, que se usan para definir factores de decaimiento aprendibles sobre las entradas y los estados ocultos. De este modo, el modelo puede aprender a “olvidar” información antigua y a imputar valores faltantes hacia medias empíricas de forma dependiente del contexto, mejorando el desempeño en tareas como clasificación de pacientes a partir de registros clínicos irregulares.

Otros trabajos relacionados incluyen:

- Modelos que concatenan directamente el *timestamp* o el intervalo de tiempo como entrada adicional a la RNN.
- Variantes de LSTM con puertas de olvido que dependen de funciones del intervalo de tiempo (por ejemplo, decaimientos exponenciales o polinomiales).
- Arquitecturas que combinan imputación explícita con RNN estándar, usando redes auxiliares para predecir valores faltantes antes de alimentarlos a la red recurrente.

En general, estos enfoques muestran que incorporar explícitamente el tiempo transcurrido entre observaciones y la estructura de datos faltantes mejora de manera consistente el rendimiento en *forecasting* y clasificación, especialmente en dominios como salud, donde la irregularidad y *missingness* son altamente informativas.

2.2.2. Modelos en tiempo continuo: Neural ODE, ODE-RNN y GRU-ODE-Bayes

Un segundo grupo de trabajos adopta una perspectiva de *tiempo continuo*, en la cual la dinámica latente de la serie se modela mediante ecuaciones diferenciales. En lugar de

actualizar el estado oculto en pasos discretos fijos, estos modelos integran una EDO entre observaciones, lo que permite manejar de manera natural tiempos de muestreo irregulares.

Entre las propuestas más destacadas se encuentran:

- **ODE-RNN y Latent ODE:** el estado latente evoluciona según una EDO parametrizada por una red neuronal (*Neural ODE*) entre observaciones, y se actualiza mediante saltos discretos al recibir nuevas mediciones. El entrenamiento se realiza mediante el método del *adjoint* continuo, permitiendo backpropagation eficiente a través del solucionador de EDO Chen et al., 2018; Yulia et al., 2019.
- **GRU-ODE-Bayes:** combina una dinámica continua basada en GRU-ODE con un tratamiento bayesiano de las observaciones, separando conceptualmente el proceso generativo continuo y el mecanismo de muestreo/observación. Esto permite manejar ruidos de medición y frecuencia de muestreo variable de manera coherente De Brouwer et al., 2019.

Estos modelos capturan con elegancia la naturaleza continua del tiempo y permiten evaluar el estado latente en cualquier instante, lo cual es atractivo para predicción multi-step y para fusionar series con distintos calendarios de muestreo. No obstante, la necesidad de resolver una EDO (o un conjunto de ellas) en cada paso introduce un costo computacional significativo, y la estabilidad numérica puede depender fuertemente de la elección del solucionador y de la parametrización de la dinámica.

2.2.3. Ecuaciones diferenciales controladas y Neural CDE

Los modelos basados en *ecuaciones diferenciales controladas* (CDE) generalizan la idea anterior modelando la trayectoria latente z_t como solución de una ecuación

$$dz_t = f_\theta(z_t) dX_t,$$

donde X_t es una versión suavizada o interpolada de la serie de entrada, considerada como un *camino control* en tiempo continuo. Las *Neural CDE* parametrizan f_θ mediante una red neuronal y resuelven la ecuación con un solucionador numérico, de forma análoga a las Neural ODE, pero interpretando las entradas como controles que impulsan la dinámica Kidger et al., 2020.

Esta formulación tiene varias ventajas para series irregulares:

- La interpolación continua de las observaciones permite manejar *cualquier calendario de muestreo*, siempre que se defina una manera razonable de rellenar los huecos entre puntos (por ejemplo, interpolación lineal o splines).
- El modelo es inherentemente *invariante a reparametrizaciones* del tiempo si se utilizan interpolaciones apropiadas, lo que lo hace robusto a cambios de resolución temporal.
- Se puede demostrar que las Neural CDE son capaces de aproximar una clase amplia de funcionales sobre caminos, lo que las hace apropiadas para tareas tanto de *forecasting* como de clasificación.

De nuevo, el costo computacional es mayor que en RNN estándar, y la complejidad de implementación (interpoladores, solucionadores, control de error) puede dificultar su adopción en entornos industriales. Además, la incorporación de mecanismos de atención similares a los de un *Transformer* no es directa en este marco.

2.3. Transformers para series de tiempo

Los *Transformers* han demostrado un rendimiento sobresaliente en procesamiento de lenguaje natural, visión por computador y, más recientemente, en forecasting de series de tiempo equiespaciadas Vaswani et al., 2017. En esta sección se revisan primero las extensiones para series regulares y, luego, las propuestas que abordan de manera explícita el muestreo irregular o el tiempo continuo.

2.3.1. Transformers para series equiespaciadas

El uso de *Transformers* en series de tiempo regulares se ha disparado en los últimos años. Numerosos trabajos han propuesto arquitecturas y componentes especializados para mejorar el rendimiento y la eficiencia computacional en pronósticos de largo horizonte, típicamente bajo el supuesto de observaciones equiespaciadas:

- **Informer**: introduce una atención *prob-sparse* para reducir la complejidad cuadrática a casi lineal en la longitud de la secuencia, manteniendo el foco en las contribuciones de atención más relevantes. Propone además un esquema de codificación temporal híbrida (posicional + *time-of-day*, *day-of-week*, etc.) y demuestra mejoras significativas en benchmarks de *forecasting* multivariante H. Zhou et al., 2021.
- **Autoformer**: replantea la auto-atención como un mecanismo de auto-correlación sobre descomposiciones tendencia-estacionalidad, proponiendo un bloque de *series decomposition* que separa componentes de baja frecuencia y patrones periódicos. Esta descomposición se integra en las capas del *Transformer*, mejorando la capacidad de modelar estructuras de largo plazo Wu et al., 2021.
- **FEDformer**: extiende la idea de Autoformer incorporando descomposición en dominios de frecuencia y empleando atención sobre componentes seleccionados en el espacio espectral, lo que reduce la carga computacional manteniendo la expresividad para series largas T. Zhou et al., 2022.

Si bien estas arquitecturas logran resultados de vanguardia en benchmarks populares, comparten supuestos clave:

- Las observaciones están en una malla *equiespaciada*, el índice de tiempo se puede tratar como un entero $1, \dots, T$.
- El *positional encoding* se define típicamente sobre estos índices discretos, a veces enriquecido con *features* calendáricas (hora del día, día de la semana, etc.), pero no sobre *timestamps* reales.

Cuando se aplican a series irregulares, estos modelos suelen requerir un preprocesamiento de re-muestreo e imputación como el descrito en la Sección 2.1, con las limitaciones asociadas.

2.3.2. Positional encodings continuos y representaciones del tiempo

Una línea de trabajo más reciente se enfoca en *representaciones continuas del tiempo* para integrarlas en arquitecturas tipo *Transformer*. Entre las propuestas relevantes se encuentran:

- **Time2Vec**: introduce una representación aprendible del tiempo que combina una componente lineal y varias componentes periódicas parametrizadas, generando un vector $v(t)$ que captura tanto tendencias como periodicidades en el tiempo. Time2Vec puede concatenarse a las entradas de un *Transformer* o de otros modelos Kazemi et al., 2019.
- Trabajos que estudian la relación entre *positional encodings discretos y continuos*, mostrando que los codificadores sinusoidales clásicos pueden interpretarse como evaluaciones discretas de funciones definidas sobre tiempo continuo, y explorando extensiones que permiten evaluar el encoding en tiempos reales intermedios. Esto abre la puerta a reemplazar el índice entero por un *timestamp* real normalizado.

Estas ideas son particularmente relevantes para series irregulares: si el *positional encoding* puede evaluarse en cualquier tiempo, es posible definir un *encoding temporal continuo* que reciba directamente los *timestamps* reales, preservando la estructura del *Transformer* original. Esta es precisamente la filosofía que se adopta en el modelo de esta memoria, descrito en los Capítulos 4 y 5.

2.3.3. Transformers sobre observaciones irregulares y modelos híbridos

Más allá de los *encodings* continuos, una línea muy relevante del estado del arte reciente consiste en construir *tokens directamente a partir de observaciones irregulares*, evitando la regularización previa sobre una malla fija. En esta familia destacan, al menos, dos estrategias representativas.

La primera consiste en tratar la serie como un *conjunto de eventos u observaciones*. Un ejemplo claro es *STraTS* Tipirneni y Reddy, 2022, que representa cada medición mediante una tripleta (t, f, v) compuesta por tiempo, variable y valor. El modelo construye un embedding inicial sumando tres componentes: un embedding de variable, un embedding continuo del valor y un embedding continuo del tiempo. Estas representaciones se contextualizan con un encoder *Transformer* y luego se fusionan mediante atención para producir un embedding global de la serie. Su contribución principal es demostrar que un esquema de atención puede operar directamente sobre datos clínicos dispersos e irregulares sin recurrir a imputación o agregación horaria agresiva. Además, incorpora un objetivo de auto-supervisión basado en *forecasting*, pero dicho objetivo se utiliza como tarea auxiliar para mejorar las representaciones, la validación principal del trabajo corresponde a clasificación clínica. Por ello, *STraTS* es cercano a esta memoria en el uso explícito de tiempos reales, pero se diferencia en que no formula el pronóstico como consultas futuras mediante tokens objetivo ni como un esquema encoder-decoder.

La segunda estrategia consiste en definir mecanismos de atención que respeten explícitamente la estructura irregular en los ejes temporal y multivariable. En esta línea, *CoFormer* Wei et al., 2023 modela cada muestra como un punto *variate-time*: una medición asociada a una variable y a un instante continuo. Primero combina un embedding de medición con códigos de variable y tiempo, luego aplica de forma sucesiva atención *intra-variate*, que conecta mediciones de la misma variable, y atención *inter-variate*, que conecta mediciones de variables distintas cercanas en el tiempo. Finalmente agrega las representaciones por variable y después a nivel de observación. Frente a un encoder *Transformer* completamente estándar, *CoFormer* introduce un sesgo inductivo más fuerte para la desalineación temporal y multivariable. Esta fortaleza depende precisamente de la existencia de varias variables

asincrónicas, en escenarios univariados, la rama intervariable pierde parte de su función original.

Junto con estos modelos orientados a observaciones, siguen existiendo propuestas que integran nociones explícitas de *tiempo continuo* en la trayectoria latente o en la representación de la secuencia:

- Modelos que combinan *Transformers* con *Neural ODE* o *Neural CDE*, utilizando la atención para resumir subsecuencias o para parametrizar el campo vectorial de la EDO/ECDE, y resolviendo luego la dinámica en tiempo continuo.
- Propuestas como el *Rough Transformer*, que se apoyan en herramientas de *rough path theory* y firmas de caminos (*signatures*) para representar trayectorias irregulares, sobre las que se aplica atención. La idea central es mapear la serie irregular a un conjunto de características invariantes a reparametrizaciones del tiempo, que luego sirven como “tokens” de entrada al *Transformer* Moreno-Pino et al., 2024.

En conjunto, estos trabajos muestran que el panorama actual ya no se limita a “transformers regulares + imputación”, sino que incluye arquitecturas capaces de operar directamente sobre observaciones asincrónicas. Sin embargo, persiste un compromiso claro entre *generalidad arquitectónica*, *sesgo inductivo específico para la irregularidad* y *simplicidad de implementación*. Precisamente ahí se sitúa la motivación de esta memoria: capturar la información temporal continua sin abandonar la estructura simple y reutilizable de un encoder *Transformer* convencional.

2.3.4. Agregación por *patches* y baselines simples para series irregulares

Muy recientemente han aparecido trabajos que replantean el *forecasting* de series irregulares desde una perspectiva de *agregación por bloques* (*patch aggregation*). Una propuesta representativa introduce un módulo de agregación consciente del tiempo, que agrupa observaciones irregulares en *patches* definidos sobre ventanas temporales y aplica operaciones de pooling o atenciones locales para producir una secuencia de tokens en una malla pseudo-regular. Esta secuencia agregada se procesa luego con redes relativamente simples (por ejemplo, MLP o bloques de atención ligeros) Liu et al., 2025.

De manera llamativa, estos métodos reportan que una combinación de:

- agregación temporal bien diseñada,
- modelos posteriores simples (incluso sin *Transformers* profundos),

puede competir o superar a arquitecturas mucho más complejas basadas en ODE/CDE o *Transformers* especializados Liu et al., 2025. Esto sugiere que, en series irregulares, *cómo se resumen los datos en el tiempo* puede ser tan importante como la arquitectura posterior.

La propuesta de esta memoria se alinea parcialmente con esta idea: en lugar de diseñar un *Transformer* radicalmente nuevo, se busca una combinación sencilla pero expresiva de:

- encoding temporal continuo sobre *timestamps* reales,
- construcción cuidadosa de ventanas de historia y *token target*,

- y un encoder *Transformer* estándar Vaswani et al., 2017.

De este modo, la irregularidad se aborda principalmente en la forma en que se representan y organizan los datos en la secuencia de entrada.

2.4. Clasificación de series de tiempo irregulares

Aunque el foco principal de esta memoria es el *forecasting*, muchas de las técnicas discutidas se han desarrollado inicialmente para tareas de *clasificación* sobre series irregulares, especialmente en el dominio de la salud. En esta sección se resume brevemente el estado del arte en clasificación, poniendo énfasis en su relación con los modelos ya descritos.

2.4.1. Métodos probabilísticos y kernels para series irregulares

Antes de la adopción masiva de deep learning, la clasificación de series irregulares se abordaba a menudo mediante:

- **Modelos de mezcla** de procesos gaussianos (por ejemplo, mezclas de GP o GP con kernels específicos por clase), donde la probabilidad de una trayectoria se evalúa bajo diferentes clases y se aplica la regla de Bayes Williams y Rasmussen, 2006.
- **Kernels dinámicos** que comparan series de distintos sujetos mediante alineamientos en tiempo continuo (por ejemplo, variantes de *dynamic time warping* continuo Müller, 2007; Sakoe y Chiba, 2003 o kernels de *time series shapelets* Hills et al., 2014; Ye y Keogh, 2009), aptos para métodos de clasificación kernelizados como SVM.

Estos enfoques explotan bien la estructura temporal y la irregularidad, pero pueden ser costosos en conjuntos de datos grandes y menos flexibles que los modelos neuronales para capturar relaciones complejas entre variables.

2.4.2. Modelos de deep learning para clasificación irregular

Buena parte de los modelos de deep learning revisados en la Sección 2.2 se introdujeron inicialmente para *clasificación* más que para *forecasting*. Ejemplos importantes incluyen:

- GRU-D y variantes con decaimientos y máscaras, aplicados a tareas como predicción de mortalidad, diagnóstico y detección temprana de sepsis Che et al., 2018.
- ODE-RNN, Latent ODE y GRU-ODE-Bayes, utilizados para modelar la evolución de estados fisiológicos latentes a partir de mediciones clínicas irregulares, con salidas de clasificación en tiempos específicos Chen et al., 2018; De Brouwer et al., 2019; Yulia et al., 2019.
- Neural CDE y modelos relacionados, que tratan la trayectoria como un camino continuo y aprenden funciones sobre dicho camino para predecir etiquetas globales o evento-tiempo Kidger et al., 2020.

En modelos que producen un embedding global de la serie, la adaptación entre clasificación y regresión puede reducirse en parte a cambiar la capa de salida y la función de pérdida. Sin embargo, el *forecasting* multi-horizonte introduce una decisión adicional:

cómo condicionar la predicción en los instantes futuros concretos. Esta distinción es relevante para esta memoria, porque CT-Transformer representa cada instante de predicción mediante un token objetivo con su timestamp, mientras que varios modelos de clasificación irregular, como STraTS y CoFormer en sus formulaciones originales, agregan primero la historia en una representación global.

2.4.3. Transformers para clasificación de series irregulares

El uso de *Transformers* para clasificación con observaciones irregulares ha madurado especialmente en el dominio biomédico, aunque la literatura sigue estando relativamente *fragmentada por tarea y dominio*. En términos generales, pueden distinguirse tres estrategias.

- **Preprocesamiento + Transformer estándar:** las series se regularizan mediante re-muestreo o imputación y se alimentan a un *Transformer* similar a los usados en series equiespaciadas, con una cabeza de clasificación sobre un token agregado.
- **Tokenización directa de observaciones irregulares:** en lugar de construir una malla temporal artificial, cada medición se convierte en un token o punto elemental. STraTS pertenece a esta familia y muestra que una arquitectura de atención aplicada a tripletes tiempo-variable-valor puede competir con baselines especializados en clasificación clínica irregular Tipirneni y Reddy, 2022. CoFormer también se sitúa en esta línea amplia, aunque con una atención más estructurada sobre vecindarios *intra/inter-variate* Wei et al., 2023.
- **Encodings temporales enriquecidos:** se mantienen los *timestamps* originales y se incorporan como encodings adicionales, por ejemplo mediante Time2Vec o embeddings sinusoidales continuos evaluados en tiempos reales Kazemi et al., 2019. Este planteamiento es el más cercano a la filosofía de la presente memoria, ya que busca preservar la forma del *Transformer* original modificando principalmente la representación temporal.

La evidencia reciente sugiere que los *Transformers* sí pueden ser competitivos en clasificación irregular cuando la representación temporal y la tokenización respetan la asincronía de las observaciones. No obstante, la mayor parte de los resultados más sólidos siguen concentrados en *clasificación clínica* y no necesariamente se trasladan de forma directa a problemas generales de *forecasting* multivariante irregular. Esta separación entre literatura de clasificación y literatura de pronóstico es una de las razones por las que sigue siendo valioso estudiar arquitecturas simples y transferibles entre ambas tareas.

2.5. Discusión y brechas de investigación

A partir de la revisión anterior, se pueden identificar varias brechas que motivan el diseño de CT-Transformer en esta memoria:

1. **Dependencia de re-muestreo e imputación.** Muchos enfoques prácticos siguen descansando en la regularización de la serie sobre una malla fija, con el riesgo de introducir sesgos y perder información sobre la distribución real de los tiempos de muestreo.

2. **Complejidad de algunos modelos en tiempo continuo.** Los modelos basados en ODE/CDE ofrecen un tratamiento elegante del tiempo continuo, pero a costa de mayor complejidad computacional y de implementación, lo que puede dificultar su uso en entornos de producción.
3. **Integración limitada entre *Transformers* y tiempo continuo para forecasting general.** Aunque existen propuestas relevantes como STraTS o CoFormer, la mayoría de los avances recientes se han validado sobre todo en clasificación irregular, frecuentemente en salud, o bien introducen mecanismos de atención muy específicos para el dominio. Siguen siendo relativamente pocas las arquitecturas que:
 - mantengan la estructura estándar de un encoder *Transformer* (auto-atención + bloques feedforward),
 - trabajen directamente con *timestamps* reales sin re-muestreo a una malla equiespaciada,
 - y permitan manejar de forma natural historias de longitud variable y horizontes de predicción heterogéneos en tareas de pronóstico multivariante.
4. **Baselines simples y comparabilidad.** Trabajos recientes orientados a series irregulares muestran que módulos de agregación temporal bien diseñados, combinados con modelos relativamente simples, pueden competir con arquitecturas profundas muy sofisticadas Liu et al., 2025. Esto pone de manifiesto la necesidad de:
 - modelos conceptualmente claros y modulares,
 - comparaciones experimentales cuidadosas frente a baselines robustos.

En este contexto, la contribución de la presente memoria se sitúa en el cruce de varias líneas:

- Diseñar un *encoding temporal continuo* que reciba directamente *timestamps* reales, inspirado en los posicional encodings sinusoidales clásicos, pero normalizado por una escala temporal configurable.
- Construir la tarea de forecasting mediante un *token objetivo* que representa explícitamente el instante de predicción dentro de la misma secuencia que la historia, permitiendo reutilizar un encoder *Transformer* estándar Vaswani et al., 2017.
- Explorar el entrenamiento con *historias de longitud variable y múltiples offsets de predicción* dentro de un mismo modelo, evaluando cómo estas decisiones de diseño afectan la capacidad de generalización en escenarios sintéticos y reales.

Los capítulos siguientes desarrollan en detalle esta propuesta: el Capítulo 3 formaliza el problema y la construcción de ejemplos supervisados, el Capítulo 4 presenta la arquitectura base de *Transformers* para series de tiempo, y el Capítulo 5 describe CT-Transformer, junto con su implementación concreta.

Capítulo 3

Series de tiempo y formulación del problema

En este capítulo se formaliza el problema que aborda esta memoria y se introducen los conceptos básicos necesarios para describirlo con precisión. El objetivo es establecer un marco formal que permita entender qué se espera de CT-Transformer y cómo se construyen los ejemplos supervisados a partir de datos temporales con marcas de tiempo no equiespaciadas.

Primero se introduce la notación para series de tiempo univariadas y multivariadas, junto con la distinción entre series equiespaciadas e irregulares. Luego se describe el problema de predicción a corto y mediano plazo como una tarea de aprendizaje supervisado, en la que el modelo recibe una *historia* (ventana de observaciones pasadas) y debe estimar el valor de la serie en uno o varios instantes futuros. Finalmente, se explica cómo se construyen los pares entrada-salida que se utilizarán durante el entrenamiento, incluyendo el rol de las longitudes de historia variables y de los offsets de predicción.

3.1. Conceptos básicos de series de tiempo

3.1.1. Definición y notación

De manera general, una *serie de tiempo* es una secuencia de observaciones ordenadas en el tiempo. En su forma más simple (caso univariado) se puede escribir como:

$$\{y_t\}_{t \in \mathcal{T}}, \quad (3.1)$$

donde $y_t \in \mathbb{R}$ es el valor observado en el tiempo t y $\mathcal{T}$ es el conjunto de instantes de observación.

En muchos problemas reales las observaciones son multivariadas. En ese caso se trabaja con vectores:

$$\{\mathbf{y}_t\}_{t \in \mathcal{T}}, \quad \mathbf{y}_t \in \mathbb{R}^{d_{\text{out}}}, \quad (3.2)$$

donde cada componente de $\mathbf{y}_t$ puede corresponder, por ejemplo, a la temperatura, presión, flujo, etc. en el mismo instante temporal.

En esta memoria será útil distinguir entre:

- **Variables de entrada** (o *features*) denotadas por $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$.
- **Variables objetivo** (o *targets*) denotadas por $\mathbf{y}_t \in \mathbb{R}^{d_{\text{out}}}$.

En algunos casos $\mathbf{x}_t$ y $\mathbf{y}_t$ pueden coincidir (se desea predecir la propia serie), mientras que en otros $\mathbf{x}_t$ y $\mathbf{y}_t$ representan magnitudes distintas.

De forma compacta, a lo largo del tiempo se dispondrá de una secuencia de tripletas:

$$\{(t_i, \mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^T, \quad (3.3)$$

con $t_1 < t_2 < \dots < t_T$ los *timestamps* asociados a cada observación.

3.1.2. Series equiespaciadas vs. no equiespaciadas

En muchos modelos clásicos de series de tiempo (por ejemplo, ARIMA o ciertas arquitecturas recurrentes) se asume que los datos están muestreados en instantes equiespaciados:

$$t_i = t_0 + i \cdot \Delta t, \quad \Delta t > 0 \text{ constante.} \quad (3.4)$$

En este caso suele ser suficiente trabajar con un índice entero i que representa el “paso de tiempo”, y buena parte de la información temporal está implícita en la posición de la observación dentro de la secuencia.

Sin embargo, en muchos dominios aplicados los datos presentan:

- **Frecuencias de muestreo variables** (sensores que cambian de tasa según el modo de operación).
- **Datos faltantes** o ventanas en las que no se registran observaciones.
- **Eventos esporádicos** que no siguen una grilla temporal fija (por ejemplo, mediciones realizadas sólo ante ciertas condiciones).

En estos casos, los instantes de observación

$$t_1, t_2, \dots, t_T \quad (3.5)$$

ya no están separados por un intervalo constante, y la información contenida en las diferencias $t_{i+1} - t_i$ pasa a ser relevante. A este tipo de datos nos referiremos como *series de tiempo no equiespaciadas* o *con timestamps irregulares*.

Trabajar directamente con timestamps reales permite, por ejemplo:

- Distinguir entre dos ejemplos con el mismo número de observaciones en la historia pero con escalas temporales distintas (historia concentrada en pocos minutos vs. varios días).
- Definir horizontes de predicción en términos de tiempo real (por ejemplo, “predecir a 15 minutos, 1 hora y 3 horas en el futuro”) en lugar de solo “a 1, 3 o 5 pasos”.

Estas características motivan la necesidad de modelos capaces de incorporar explícitamente la información temporal continua, lo que se abordará en capítulos posteriores mediante mecanismos de *encoding temporal continuo* dentro de la arquitectura Transformer.

3.2. Datos considerados en la memoria

La memoria considera dos escenarios generales de datos:

- **Datos sintéticos**, utilizados como entorno controlado para validar el comportamiento básico del pipeline y de la arquitectura en un contexto donde la generación de la serie puede analizarse con mayor trazabilidad.
- **Datos reales procesados**, organizados como una colección de datasets temporales independientes, sobre los cuales se ejecuta el benchmark principal reportado en los capítulos experimentales.

En ambos casos, la estructura de trabajo es esencialmente la misma: cada dataset contiene una columna temporal que fija el orden de observación y una o más variables numéricas que actúan como entradas y objetivos de predicción. En la etapa actualmente consolidada de esta memoria, el benchmark principal se apoya sobre datasets reales procesados con una variable objetivo escalar, de modo que el problema experimental activo corresponde a forecasting univariado con timestamps irregulares.

La razón para considerar tanto datos sintéticos como reales es metodológica. Los datos sintéticos permiten verificar que el pipeline de construcción de ventanas, escalado, entrenamiento y predicción funciona de extremo a extremo bajo condiciones más controladas. Los datos reales, en cambio, permiten evaluar si la arquitectura mantiene un comportamiento competitivo en escenarios más cercanos a una aplicación práctica, donde la irregularidad temporal y la variabilidad entre datasets son parte central del problema.

Los detalles específicos del protocolo experimental, de las configuraciones de entrenamiento y de la comparación entre modelos se desarrollan en el Capítulo 6. En este capítulo interesa principalmente dejar establecido que la formulación matemática introducida a continuación busca cubrir ambos escenarios bajo un mismo marco general.

3.3. Problema de predicción en series de tiempo

3.3.1. Predicción a partir de una ventana de historia

En la práctica, el problema de predicción se formula como una tarea de aprendizaje supervisado sobre ventanas finitas de la serie. Sea

$$\mathcal{H}_i = \{(t_j, \mathbf{x}_j, \mathbf{y}_j) : j \leq i\} \quad (3.6)$$

la historia completa disponible hasta el índice i . Dado un horizonte de predicción definido por un instante futuro $t^* > t_i$, se desea construir un modelo

$$F_\theta : \mathcal{H}_i \mapsto \hat{\mathbf{y}}_{t^*}, \quad (3.7)$$

parametrizado por θ , que entregue una predicción $\hat{\mathbf{y}}_{t^*}$ de la variable objetivo en t^* .

Por motivos computacionales y de regularización, se trabaja con una *historia truncada* de longitud acotada. En esta memoria se utiliza una ventana deslizante que, para cada ejemplo, considera sólo las últimas h observaciones antes de t^* :

$$\mathcal{H}_i^{(h)} = \{(t_j, \mathbf{x}_j, \mathbf{y}_j) : i - h + 1 \leq j \leq i\}. \quad (3.8)$$

En otras palabras, el modelo verá únicamente un fragmento reciente de la serie y deberá aprender a extraer de allí la información relevante para predecir en el horizonte deseado.

3.3.2. Horizontes de predicción y offsets

En el caso de datos equiespaciados, es habitual definir el horizonte de predicción en términos de pasos discretos, por ejemplo:

$$\text{predecir } y_{t+k} \text{ a partir de } \{y_{t-h+1}, \dots, y_t\}, \quad (3.9)$$

donde k es el número de pasos hacia adelante.

En esta memoria se adopta una formulación que combina ambas visiones:

- A nivel de construcción del dataset, se define un *offset de predicción* o que indica cuántos índices después del *anchor* se tomará el target en la secuencia original.
- A nivel del modelo, además, se utilizan los timestamps reales t_i para construir tiempos relativos respecto del primer timestamp válido de la ventana. Así, tanto las observaciones históricas como el instante objetivo se expresan en una escala común, la distancia temporal entre la última observación de la historia y el objetivo queda disponible como diferencia entre sus tiempos relativos, no como un cambio del origen temporal.

Más concretamente, para cada ejemplo se elige un índice *anchor* a (que marca el final de la historia) y un offset de predicción $o \geq 0$. El índice del target será entonces:

$$i_{\text{target}} = a + o. \quad (3.10)$$

El instante objetivo correspondiente es $t_{i_{\text{target}}}$ y el valor a predecir es $\mathbf{y}_{i_{\text{target}}}$.

Durante el entrenamiento se pueden considerar:

- Un único offset fijo (por ejemplo, siempre predecir un paso adelante).
- Un conjunto discreto de offsets posibles (por ejemplo $\{1, 3, 5\}$), que se muestren aleatoriamente. Esto permite entrenar al modelo para manejar distintos horizontes dentro de un mismo esquema.

3.4. Construcción de ejemplos supervisados

3.4.1. Ventanas deslizantes y tamaño de la historia

Sea T el número total de observaciones disponibles. Dado un *tamaño máximo de historia* $h_{\text{máx}}$ y un *stride* $s \geq 1$, se generan ejemplos recorriendo la serie con una ventana deslizante. De manera simplificada, los índices *anchor* se pueden escribir como:

$$a \in \{h_{\text{máx}}, h_{\text{máx}} + s, h_{\text{máx}} + 2s, \dots, a_{\text{máx}}\}, \quad (3.11)$$

donde $a_{\text{máx}}$ se elige de modo que exista al menos un offset de predicción válido (es decir, $a_{\text{máx}} + o_{\text{máx}} \leq T - 1$, con $o_{\text{máx}}$ el máximo offset considerado).

Para cada anchor a se define una historia formada por las últimas h observaciones anteriores a a :

$$\mathcal{H}_a^{(h)} = \{(t_j, \mathbf{x}_j, \mathbf{y}_j)\}_{j=a-h}^{a-1}, \quad (3.12)$$

donde h puede tomar valores entre una longitud mínima $h_{\text{mín}}$ y la longitud máxima $h_{\text{máx}}$:

$$h_{\text{mín}} \leq h \leq h_{\text{máx}}. \quad (3.13)$$

En esta memoria se consideran dos configuraciones:

- **Historia fija:** $h_{\text{mín}} = h_{\text{máx}}$, por lo que todas las historias tienen la misma longitud.
- **Historia variable:** $h_{\text{mín}} < h_{\text{máx}}$, y para cada ejemplo se muestrea aleatoriamente una longitud $h \in [h_{\text{mín}}, h_{\text{máx}}]$. De este modo, el modelo se entrena para manejar ventanas de contexto de tamaño variable, algo relevante cuando la densidad temporal de los datos cambia en el tiempo.

3.4.2. Definición de la entrada y la salida de cada ejemplo

Cada ejemplo supervisado queda entonces determinado por:

$$\text{Entrada} : \left(\{t_j\}_{j=a-h}^{a-1}, \{\mathbf{x}_j\}_{j=a-h}^{a-1}, \{\mathbf{y}_j\}_{j=a-h}^{a-1}, t_{i_{\text{target}}} \right), \quad (3.14)$$

$$\text{Salida} : \mathbf{y}_{i_{\text{target}}}, \quad (3.15)$$

donde $i_{\text{target}} = a + o$ como se describió antes.

En términos más operativos, es útil separar:

- Un vector de tiempos de la historia:

$$\mathbf{t}^{(\text{past})} = (t_{a-h}, \dots, t_{a-1}),$$

- Una matriz de valores de entrada:

$$X^{(\text{past})} \in \mathbb{R}^{h \times d_{\text{in}}},$$

- El timestamp objetivo $t^{(\text{target})} = t_{i_{\text{target}}}$,

- Y el vector de salida

$$\mathbf{y}^{(\text{target})} = \mathbf{y}_{i_{\text{target}}} \in \mathbb{R}^{d_{\text{out}}}.$$

La familia CT-Transformer recibirá como entrada una representación construida a partir de la historia y del timestamp objetivo. En la arquitectura base esto se realiza mediante la adición de tokens objetivo a la misma secuencia de entrada, en la variante encoder-decoder, esos tokens se procesan en el decoder y consultan la historia codificada por el encoder. En ambos casos, el modelo deberá producir una estimación $\hat{\mathbf{y}}^{(\text{target})}$ de la salida verdadera. Los detalles de esta representación se desarrollan en el capítulo dedicado al modelo Transformer.

3.5. Formulación como problema de aprendizaje supervisado

3.5.1. Función objetivo

Sea $\mathcal{D}_{\text{train}}$ el conjunto de ejemplos contruidos según el procedimiento anterior. El problema de entrenamiento consiste en hallar parámetros θ que minimicen una función de pérdida agregada sobre todos los ejemplos de entrenamiento.

Para problemas de regresión se utilizará principalmente la pérdida de error cuadrático medio:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\text{entrada}, \mathbf{y}) \in \mathcal{D}_{\text{train}}} \|F_{\theta}(\text{entrada}) - \mathbf{y}\|^2, \quad (3.16)$$

donde F_{θ} representa al modelo (en este caso, el Transformer especializado) y la norma se aplica sobre las componentes de la variable objetivo.

En el capítulo de entrenamiento se considerarán también otras métricas de evaluación, como error absoluto medio (MAE), raíz del error cuadrático medio (RMSE) y error porcentual absoluto medio (MAPE), que permiten interpretar la calidad de las predicciones en distintas unidades y escalas.

3.5.2. Entrenamiento, validación y prueba

Para estimar la capacidad de generalización del modelo —es decir, su desempeño sobre datos no vistos durante el entrenamiento— se divide la serie en tres subconjuntos temporales disjuntos:

- **Entrenamiento:** segmento inicial de la serie, usado para ajustar los parámetros del modelo.
- **Validación:** segmento intermedio, usado para seleccionar hiperparámetros (por ejemplo, tamaño de historia, offsets de predicción, arquitectura del modelo) y para aplicar criterios de *early stopping*.
- **Prueba:** segmento final, reservado para evaluar el desempeño del modelo una vez fijados los hiperparámetros.

La división se realiza respetando el orden temporal de los datos, de modo que el modelo siempre se entrene en el pasado y se evalúe en el futuro, evitando fugas de información temporal.

3.6. Resumen del capítulo

En este capítulo se han establecido los elementos necesarios para describir con precisión el problema abordado en la memoria:

- Se introdujo la notación básica para series de tiempo univariadas y multivariadas, distinguiendo explícitamente entre variables de entrada $\mathbf{x}_t$, variables objetivo $\mathbf{y}_t$ y timestamps t .

- Se discutió la diferencia entre series equiespaciadas y no equiespaciadas, destacando las motivaciones para trabajar directamente con marcas de tiempo irregulares.
- Se describió, a nivel general, el tipo de datos considerados en la memoria, distinguiendo entre escenarios sintéticos de validación controlada y datasets reales procesados usados en el benchmark principal.
- Se formuló el problema de predicción como una tarea de aprendizaje supervisado basada en ventanas de historia finita y horizontes de predicción definidos mediante offsets sobre la secuencia original y timestamps reales.
- Se describió el procedimiento para construir ejemplos supervisados a partir de la serie completa, incluyendo la posibilidad de usar longitudes de historia variables y un conjunto discreto de offsets de predicción.
- Se presentó la función objetivo de entrenamiento y el esquema de partición temporal en conjuntos de entrenamiento, validación y prueba.

Sobre este marco se construirá, en los capítulos siguientes, la arquitectura CT-Transformer para manejar timestamps no equiespaciados y realizar predicciones a distintos horizontes, junto con los experimentos que evalúan su desempeño.

Capítulo 4

Transformers para series de tiempo no equiespaciadas

En este capítulo se presenta el marco conceptual que conecta la arquitectura *Transformer* con el problema de series de tiempo no equiespaciadas abordado en esta memoria. El objetivo es fijar los componentes del Transformer clásico que se preservan, explicitar por qué los timestamps irregulares obligan a adaptar la representación temporal y dejar establecido el conjunto de decisiones de diseño que luego se concretarán en CT-Transformer.

Primero se revisa brevemente la arquitectura Transformer clásica como modelo de atención sobre secuencias. A continuación se discuten sus ventajas y desafíos al trabajar con series de tiempo. Finalmente, se resumen los principios de adaptación que motivan CT-Transformer, dejando la especificación completa del modelo para el capítulo siguiente.

4.1. Recordatorio de la arquitectura Transformer

El Transformer Vaswani et al., 2017 es una arquitectura originalmente propuesta para tareas de procesamiento de lenguaje natural, donde la entrada es una secuencia de *tokens* discretos. Cada token x_i se proyecta a un vector denso $\mathbf{e}_i \in \mathbb{R}^{d_{\text{model}}}$ mediante una tabla de embeddings, y se le suma un *positional encoding* que entrega información sobre su posición en la secuencia. Sobre esta representación se aplican varias capas de:

- **Atención multi-cabeza** (*Multi-Head Self-Attention*), que mezcla información entre todas las posiciones.
- **Bloques feedforward** independientes por posición.
- **Conexiones residuales** y normalización de capa (*LayerNorm*).

En cada capa, la atención utiliza tres proyecciones lineales de los vectores de entrada:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (4.1)$$

donde $X \in \mathbb{R}^{L \times d_{\text{model}}}$ es la matriz de representaciones de la secuencia, L es la longitud de la secuencia y W^Q, W^K, W^V son matrices de pesos. La atención escalar producto se computa como:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V, \quad (4.2)$$

donde d_h es la dimensión de cada cabeza. En la versión multi-cabeza se repite este cálculo en paralelo para varias proyecciones y se concatenan los resultados.

En esta memoria se consideran dos formulaciones basadas en Transformer. La formulación principal, utilizada por CT-Transformer base y sus variantes más directas, emplea un *encoder Transformer* que recibe una secuencia de vectores ya embebidos, aplica varias capas de auto-atención multi-cabeza y redes feedforward, y devuelve una representación contextualizada para cada posición de la secuencia. Además, el benchmark incluye variantes encoder-decoder, donde la historia se procesa con este mismo tipo de encoder y los timestamps objetivo se procesan en un decoder mediante auto-atención multi-cabeza y atención cruzada multi-cabeza. Por claridad, este capítulo desarrolla primero la formulación basada en encoder, y luego explicita cómo se diferencia la extensión encoder-decoder.

4.2. Transformers en series de tiempo

4.2.1. Ventajas

Aplicar Transformers a series de tiempo presenta varias ventajas:

- **Dependencias de largo alcance:** la auto-atención permite que cualquier posición atienda directamente a cualquier otra, independientemente de la distancia temporal, lo que facilita capturar patrones de largo plazo.
- **Paralelización:** a diferencia de modelos autoregresivos estrictos (como ciertas redes recurrentes), todas las posiciones de la historia pueden procesarse en paralelo, lo que resulta eficiente en hardware moderno.
- **Flexibilidad en la longitud de la secuencia:** el mismo conjunto de pesos puede operar sobre secuencias de longitud variable, siempre que la dimensión de los vectores de entrada se mantenga constante.

Estas características son especialmente atractivas para problemas donde las ventanas de historia pueden tener tamaños distintos y donde se desea manejar horizontes de predicción múltiples, como se describió en el Capítulo 3.

4.2.2. Desafíos con timestamps irregulares

Los Transformers clásicos asumen que la información de *posición* se puede representar mediante índices enteros $1, 2, \dots, L$, y utilizan un encoding posicional (típicamente sinusoidal) definido sobre esos índices. Sin embargo, en series de tiempo no equiespaciadas:

- La separación temporal entre observaciones consecutivas, $t_{i+1} - t_i$, no es constante.
- Puede haber lagunas de observaciones o períodos de muestreo más denso.
- El horizonte de predicción puede definirse naturalmente en unidades de tiempo físico (segundos, minutos, horas) y no sólo en “pasos” discretos.

Si se ignorara esta irregularidad y se tratara cada observación como si correspondiera a un paso fijo, el modelo podría confundir historias que, pese a tener el mismo número de puntos, se desarrollan en escalas de tiempo muy distintas. Por ello, en esta memoria se propone incorporar explícitamente el timestamp real en el encoding temporal del Transformer.

4.3. Principios de adaptación para series no equiespaciadas

A partir de las ventajas y limitaciones descritas, adaptar un Transformer al problema de series no equiespaciadas exige intervenir principalmente en la *representación de la secuencia*, no en el bloque encoder en sí. En esta memoria, esa adaptación se apoya en cuatro decisiones de diseño.

1. **Usar timestamps reales en lugar de posiciones discretas.** La posición de cada observación deja de interpretarse como un índice entero y pasa a representarse mediante un encoding temporal evaluado directamente sobre el tiempo físico.
2. **Representar la predicción como una consulta explícita.** En vez de resumir toda la historia en un embedding global y luego proyectarlo al futuro, la arquitectura incorpora tokens objetivo que representan los instantes donde se desea predecir.
3. **Separar valor, tiempo y rol.** Cada token combina la información observada, su timestamp y un identificador que distingue historia y objetivo, de modo que el modelo no confunda observaciones ya vistas con consultas futuras.
4. **Mantener el encoder Transformer como núcleo computacional.** La irregularidad temporal se incorpora en la representación inicial de los tokens, mientras que la atención multi-cabeza, las conexiones residuales y los bloques feedforward permanecen esencialmente sin cambios.

En términos conceptuales, estas decisiones pueden resumirse en la representación inicial

$$\mathbf{h}_i^{(0)} = E_{\text{val}}(x_i) + E_{\text{time}}(t_i) + E_{\text{rol}}(f_i), \quad (4.3)$$

donde x_i representa el valor asociado al token, t_i su timestamp y f_i su rol dentro de la secuencia. La ecuación no fija todavía una implementación concreta, sólo explicita que la adaptación al tiempo irregular ocurre a nivel de embeddings y construcción de tokens.

4.4. Del marco conceptual al modelo propuesto

Sobre esta base, el paso desde el Transformer clásico hacia CT-Transformer consiste en especificar una serie de decisiones concretas que ya pertenecen al modelo propuesto propiamente tal:

- cómo se construyen ejemplos supervisados con historias variables y offsets de predicción,
- cómo se agregan tokens objetivo al final de la historia,
- cómo se normaliza el tiempo y qué variante de encoding temporal se utiliza,
- cómo se organiza la variante encoder-decoder respecto de la arquitectura base de encoder único,
- y cómo se formulan la pérdida, la inferencia y el pronóstico multi-step en la implementación experimental.

Estas decisiones ya no son parte de un recordatorio general sobre Transformers, sino de la definición específica de CT-Transformer. Por eso se desarrollan en el Capítulo 5, donde se describe la arquitectura completa, su representación de entrada y sus variantes experimentales.

Conviene subrayar la división de responsabilidades entre ambos capítulos. El presente capítulo responde a la pregunta *por qué* un Transformer estándar necesita ser adaptado cuando el tiempo deja de ser equiespaciado. El capítulo siguiente responde a *cómo* se implementa esa adaptación en la arquitectura concreta evaluada en la memoria.

4.5. Resumen del capítulo

En este capítulo se ha fijado el marco conceptual que conecta la arquitectura Transformer con el problema de series de tiempo no equiespaciadas. Los elementos clave son:

- El bloque encoder Transformer sigue siendo el núcleo del modelo, la adaptación principal ocurre en la forma de representar la secuencia de entrada.
- El uso de timestamps irregulares obliga a reemplazar la noción de posición discreta por una representación temporal continua evaluada en tiempos reales.
- La predicción puede formularse como una consulta explícita mediante tokens objetivo, en lugar de depender únicamente de un embedding global de la historia.
- La separación entre valor, tiempo y rol permite conservar la estructura general del Transformer e introducir información temporal sin rediseñar por completo la arquitectura.
- La especificación completa de estas decisiones define ya a CT-Transformer y se desarrolla en el capítulo siguiente.

En el capítulo siguiente se describe CT-Transformer en detalle: construcción de ejemplos, secuencias con tokens objetivo, embeddings, variantes arquitectónicas e implicancias para entrenamiento e inferencia.

Capítulo 5

CT-Transformer para series de tiempo no equiespaciadas

En este capítulo se describe en detalle la familia de modelos Continuous-Time Transformer (CT-Transformer) propuesta para abordar la predicción de series de tiempo *no equiespaciadas* utilizando arquitecturas tipo Transformer. A diferencia del capítulo anterior, aquí ya no se discute el problema en términos conceptuales generales, sino que se fija la especificación concreta del modelo evaluado en la memoria: cómo se construyen las entradas, cómo se representan historia y objetivos, qué variantes arquitectónicas se consideran y cómo estas decisiones se traducen en entrenamiento e inferencia.

Recordando la notación del Capítulo 3, consideremos una serie temporal multivariada observada en instantes

$$t_1 < t_2 < \dots < t_T, \quad t_i \in \mathbb{R},$$

no necesariamente equiespaciados, y valores observados

$$x_i = x(t_i) \in \mathbb{R}^D, \quad i = 1, \dots, T.$$

El objetivo de CT-Transformer es aprender una función

$$f_\theta : \left(\{(t_{i-h+1}, x_{i-h+1}), \dots, (t_i, x_i)\}, t^* \right) \mapsto \hat{y}(t^*),$$

capaz de predecir el valor futuro (o pasado) en un *timestamp* objetivo t^* , a partir de una historia de longitud variable h .

La arquitectura base de CT-Transformer utilizada en esta memoria combina:

- un *embedding* de valores continuos,
- un *encoding temporal continuo* basado en *timestamps* reales,
- un identificador aprendible para distinguir tokens de historia y tokens objetivo,
- un *encoder Transformer* estándar sobre la secuencia enriquecida,
- y una cabeza de regresión sobre los tokens objetivo.

Además de esta arquitectura base, el benchmark incluye variantes encoder-decoder que reutilizan los mismos componentes de representación, pero separan la historia y los objetivos en dos flujos: la historia se codifica en el encoder y los tokens objetivo se procesan en el decoder mediante atención cruzada. En las secciones siguientes se detalla primero la arquitectura base y luego se describe la variante encoder-decoder.

5.1. Objetivos de diseño

CT-Transformer se construyó alrededor de los siguientes objetivos de diseño:

1. **Respetar la irregularidad temporal:** evitar, en la medida de lo posible, el re-muestreo a una malla equiespaciada, trabajando directamente con los *timestamps* originales.
2. **Separar claramente valor y tiempo:** representar por separado las variables de entrada (temperatura, presión, etc.) y la información temporal, combinándolas recién a nivel de *embedding*.
3. **Mantener compatibilidad con el Transformer estándar:** reutilizar la mayor parte de la estructura de atención multi-cabeza y bloques encoder descritos en el Capítulo 4, de modo que las extensiones sean lo menos invasivas posible.
4. **Manejar historias de longitud variable y horizontes de predicción heterogéneos:** permitir que, durante el entrenamiento, el modelo vea ejemplos con distintas longitudes de historia y con diferentes *offsets de predicción*, con el fin de mejorar su robustez.
5. **Facilitar la inferencia multi-step:** permitir predicciones para uno o varios instantes futuros mediante una misma interfaz, delegando al pipeline la construcción de los tokens objetivo, la actualización de la ventana de historia y la conversión de las predicciones a la escala original.

5.2. Representación de entradas y construcción de ejemplos

5.2.1. Separación entre valores y timestamps

A nivel de datos crudos, el modelo recibe:

- un arreglo de *timestamps* $(t_1, \dots, t_T)$ ya ordenado,
- una matriz de valores $V \in \mathbb{R}^{T \times D_{\text{total}}}$ que contiene:
 - las primeras D_{in} columnas como variables de entrada (*features*),
 - las siguientes D_{out} como variables objetivo (*targets*), cuando no se especifica un arreglo de *targets* separado.

Esta organización se materializa en la etapa de construcción de ejemplos supervisados, que recibe:

- una matriz de valores $[T, D_{\text{total}}]$,
- un vector de timestamps $[T]$,
- las dimensiones de entrada y salida para separar las variables usadas como contexto de las variables objetivo.

El preprocesamiento descrito en el Capítulo 3 (normalización ajustada sólo con el bloque de entrenamiento, particiones temporales, etc.) se aplica antes de construir dichos ejemplos.

5.2.2. Ventanas de historia y targets

Cada ejemplo supervisado se construye a partir de cuatro elementos clave:

1. **Longitud máxima de historia** ($h_{\text{máx}}$): número máximo de puntos pasados considerados.
2. **Longitud mínima de historia** ($h_{\text{mín}}$): permite muestrear historias de longitud variable entre $h_{\text{mín}}$ y $h_{\text{máx}}$.
3. **Offsets de predicción**: distancias en índices entre un *anchor* interno a y los índices objetivo.
4. **Número de objetivos por ejemplo**: cantidad de timestamps futuros distintos que se predicen simultáneamente.

Para cada ejemplo se elige un índice *anchor* a tal que exista suficiente historia hacia atrás y suficiente espacio hacia adelante para el target. En la construcción implementada, a se interpreta como la frontera derecha de la historia: la historia termina en el índice $a - 1$ y los objetivos se ubican en índices $a + o_m$. Por tanto, usando índices internos base cero:

$$a \in \{h_{\text{máx}}, h_{\text{máx}} + s, \dots, T - 1 - o_{\text{máx}}\},$$

donde s es el salto de la ventana y $o_{\text{máx}}$ es el máximo offset posible.

Dado un *anchor* a :

- se muestrea una longitud de historia $h \in [h_{\text{mín}}, h_{\text{máx}}]$ (si la longitud mínima es distinta de la longitud máxima, de lo contrario $h = h_{\text{máx}}$),
- se define la historia como

$$\{(t_{a-h}, x_{a-h}), \dots, (t_{a-1}, x_{a-1})\},$$

- se elige un conjunto ordenado de offsets de predicción $\{o_1, \dots, o_M\}$ y se definen los índices objetivo

$$j_m = a + o_m, \quad m = 1, \dots, M,$$

- los timestamps objetivo son $t_m^* = t_{j_m}$ y los valores objetivo son $y_m^* = y_{j_m}$.

El resultado conceptual de cada ejemplo es:

- valores pasados: matriz $[h, D_{\text{in}}]$,
- timestamps pasados: vector $[h]$,
- timestamps objetivo: vector $[M]$,
- valores objetivo: matriz $[M, D_{\text{out}}]$.

Esta representación todavía no está en el formato que consume el Transformer, para ello se introduce una etapa de construcción de secuencias.

5.2.3. Construcción de secuencias con token objetivo

La construcción de secuencias transforma el ejemplo anterior en la entrada final del modelo. Conceptualmente, se añaden uno o más *tokens objetivo* al final de la historia:

$$\underbrace{(t_{a-h}, x_{a-h}), \dots, (t_{a-1}, x_{a-1})}_{\text{historia}}, \underbrace{(t_1^*, \tilde{x}_1^*), \dots, (t_M^*, \tilde{x}_M^*)}_{\text{tokens objetivo}}$$

donde $\tilde{x}_m^*$ es un *placeholder* para los valores de entrada del token objetivo. Este vector no corresponde al valor real futuro que se quiere predecir, dicho valor se mantiene separado y sólo se utiliza para calcular la pérdida durante el entrenamiento y las métricas durante la evaluación. El constructor de secuencias admite dos formas de inicializar este placeholder:

- **Ceros:** $\tilde{x}_m^* = \mathbf{0}$ para cada token objetivo. Esta es la opción utilizada en los experimentos principales de esta memoria. En este caso, el token objetivo no recibe información de valor observada en el instante futuro, queda definido por su rol de objetivo y por su timestamp t_m^* .
- **Último valor observado:** $\tilde{x}_m^*$ copia el último vector de entrada disponible en la historia. Esta alternativa entrega al token objetivo una señal tipo persistencia, pero no introduce el valor verdadero del objetivo. En la representación densa corresponde a copiar el último vector multivariado observado, en la representación por eventos debe interpretarse sólo como una inicialización alternativa del valor del token de consulta, por lo que no se utiliza como configuración principal.

Por lo tanto, en la configuración reportada en el benchmark, los tokens objetivo no contienen ni una imputación del futuro ni una copia del valor que se desea predecir. La predicción se apoya en la historia observada, en el identificador de rol historia-objetivo y en el encoding temporal continuo aplicado al timestamp objetivo.

Después de esta operación, la entrada al modelo contiene:

- valores de entrada: matriz $[L, D_{\text{in}}]$ con $L = h + M$ (historia + tokens objetivo),
- timestamps de entrada: vector $[L]$ con los *timestamps* de historia y $\{t_1^*, \dots, t_M^*\}$,
- máscara de rol: vector booleano $[L]$ activo sólo en las posiciones objetivo,
- valores objetivo y timestamps objetivo, usados para calcular la pérdida y reportar métricas.

La idea central es que cada token objetivo queda definido por la combinación de una máscara que identifica su rol y de su *timestamp*. La máscara sólo indica que esa posición debe interpretarse como una consulta de predicción, el timestamp fija la fecha concreta a la que apunta esa consulta. Dicho de otro modo, el horizonte de predicción no se controla con un identificador discreto adicional, sino asignando a cada token objetivo su timestamp t_m^* y dejando que el encoding temporal continuo transforme ese tiempo en una representación vectorial.

Posteriormente, la etapa de agrupación por lotes realiza el *padding* necesario cuando las longitudes L de las secuencias del batch difieren.

5.3. Arquitectura de CT-Transformer

La arquitectura base de CT-Transformer puede entenderse como un encoder Transformer enriquecido con información temporal continua. A grandes rasgos, el flujo interno es:

1. Proyección de valores continuos a la dimensión del modelo.
2. Cálculo del encoding temporal continuo.
3. Embedding del identificador que distingue historia y objetivo.
4. Suma de las tres contribuciones para formar los *tokens* de entrada.
5. Paso por una pila de bloques encoder Transformer.
6. Extracción de los estados de los tokens objetivo y paso por una cabeza de regresión.

5.3.1. Embedding de valores continuos

El embedding de valores implementa una proyección lineal de las variables de entrada:

$$e_t^{(\text{val})} = W_{\text{val}}x_t + b_{\text{val}}, \quad x_t \in \mathbb{R}^{D_{\text{in}}}, e_t^{(\text{val})} \in \mathbb{R}^{d_{\text{model}}}.$$

Para un batch de tamaño B y longitud de secuencia L , la entrada tiene forma $[B, L, D_{\text{in}}]$ y la salida $[B, L, d_{\text{model}}]$. Opcionalmente puede aplicarse una normalización de capa sobre la dimensión d_{model} tras la proyección.

5.3.2. Encoding temporal continuo

El componente clave para manejar *timestamps* irregulares es el encoding temporal continuo. En lugar de asumir posiciones enteras $\{0, 1, 2, \dots\}$, se trabaja directamente con los tiempos reales. Para cada secuencia en el batch se calcula primero un tiempo relativo:

$$\tau_t = \frac{t - t_0}{s_{\text{time}}},$$

donde t_0 es el primer *timestamp* de la secuencia y s_{time} es un factor de escala temporal (por ejemplo, 900 segundos para que un paso unitario corresponda a 15 minutos).

En la configuración base se aplica además una compresión $\log(1 + \tau_t)$, restringiendo previamente τ_t a valores no negativos. Esta transformación hace más robusta la representación cuando existen horizontes largos o huecos temporales grandes, ya que evita que los argumentos sinusoidales crezcan de forma desproporcionada. En las variantes Time2Vec del benchmark se evalúa también la versión lineal de τ_t para aislar el efecto de una representación temporal aprendida.

La elección de trabajar con τ_t en lugar de t crudo introduce una normalización explícita de escala, lo que ayuda a que el modelo se enfoque en *estructuras temporales relativas* más que en valores absolutos de tiempo. Esta decisión es especialmente importante en el token objetivo: al cambiar t^* , cambia también τ_{target} y, por ende, cambia el vector $e_{\text{target}}^{(\text{time})}$ incluso si el token sigue ocupando la última posición de la secuencia. Así, el modelo puede distinguir entre “predecir dentro de 15 minutos” y “predecir dentro de 3 horas” sin modificar la arquitectura del encoder.

Modo sinusoidal (fijo)

En el modo sinusoidal, para cada dimensión k del embedding temporal se define:

$$\text{PE}_{t,2k} = \sin\left(\frac{\tau_t}{\omega_k}\right), \quad \text{PE}_{t,2k+1} = \cos\left(\frac{\tau_t}{\omega_k}\right),$$

donde las escalas temporales ω_k se construyen de manera análoga al *positional encoding* original de Vaswani et al., 2017. En particular,

$$\omega_k = 10000^{\frac{2k}{d_{\text{model}}}}, \quad k = 0, \dots, \left\lfloor \frac{d_{\text{model}}}{2} \right\rfloor - 1.$$

Así, las dimensiones $2k$ y $2k + 1$ comparten la misma escala ω_k , pero usan funciones trigonométricas distintas. Esto produce un vector $e_t^{(\text{time})} \in \mathbb{R}^{d_{\text{model}}}$ que codifica de forma suave la posición relativa en el tiempo.

Si bien este encoding es simple y no requiere parámetros entrenables, presenta una limitación importante: las escalas ω_k son fijas y están determinadas por la fórmula del Transformer original, sin adaptarse a la escala temporal específica de cada conjunto de datos. Además, la elección del factor de escala s_{time} introduce un hiperparámetro sensible que debe calibrarse manualmente.

La Figura 5.1 ilustra esta diferencia usando el mismo rango de tiempo relativo en ambos paneles. En el encoding tradicional, las filas avanzan según una grilla de posiciones discretas, en el encoding temporal continuo, en cambio, el mismo intervalo puede evaluarse en coordenadas temporales intermedias, mostrando que la representación no queda restringida a índices enteros de posición.

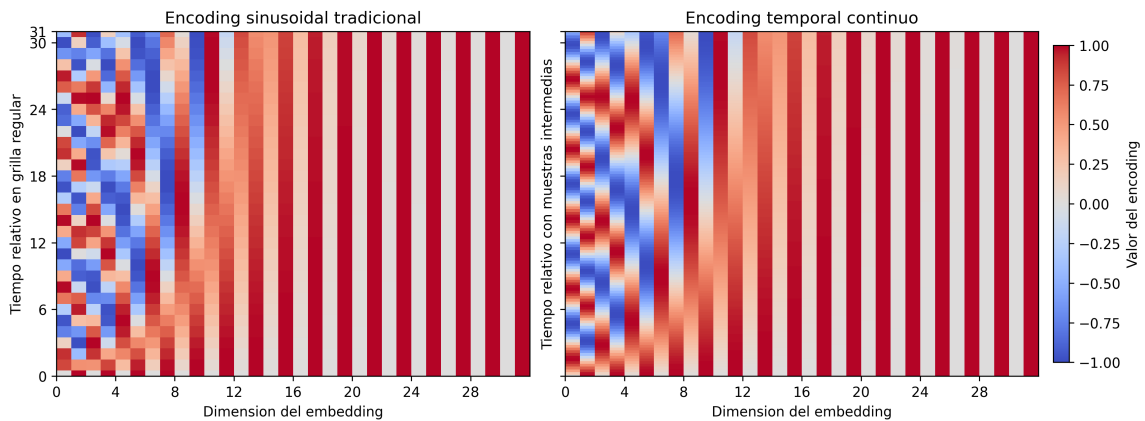


Figura 5.1: Comparación entre el encoding sinusoidal tradicional y el encoding temporal continuo. Ambos paneles cubren el mismo rango de tiempo relativo y usan la misma familia de funciones seno y coseno, la diferencia está en la coordenada de entrada: una grilla discreta de posiciones en el caso tradicional y una grilla temporal más densa, con valores intermedios, en el caso continuo.

Modo Time2Vec (aprendido)

Para superar las limitaciones del encoding sinusoidal fijo, se implementa una variante basada en Time2Vec Kazemi et al., 2019, que reemplaza las frecuencias fijas por componentes

aprendidos durante el entrenamiento. La formulación es:

$$\text{Time2Vec}(\tau)[i] = \begin{cases} \omega_0 \cdot \tau + \varphi_0, & \text{si } i = 0 \quad (\text{componente lineal}), \\ \sin(\omega_i \cdot \tau + \varphi_i), & \text{si } i > 0 \quad (\text{componentes periódicos}), \end{cases}$$

donde los pesos ω_i y las fases φ_i son parámetros entrenables. El resultado es un vector $e_t^{(\text{time})} \in \mathbb{R}^{d_{\text{model}}}$, compuesto por:

- un componente lineal ($i = 0$) que captura tendencias aperiódicas y derivas temporales,
- $d_{\text{model}} - 1$ componentes periódicos que capturan patrones cíclicos a frecuencias *aprendidas* del dato.

En la implementación actual el componente lineal se calcula en precisión de 32 bits y la salida completa se normaliza por capa. Esta decisión conserva la señal lineal de Time2Vec, evita saturarla artificialmente con una función acotada y mejora la estabilidad bajo precisión mixta durante el entrenamiento.

A diferencia del modo sinusoidal, Time2Vec presenta tres ventajas concretas:

1. Las frecuencias ω_i se aprenden durante el entrenamiento, adaptándose automáticamente a la escala temporal de cada dataset.
2. Las fases φ_i permiten representar desfases temporales que el encoding sinusoidal fijo no puede capturar.
3. El componente lineal captura tendencias monótonas que las funciones periódicas por sí solas no pueden modelar.

El sesgo inductivo periódico (función seno) previene el sobreajuste que podría ocurrir con un MLP completamente libre, mientras que la capacidad de aprendizaje permite al modelo descubrir las escalas temporales relevantes sin intervención manual.

La Figura 5.2 muestra una visualización cualitativa de esta diferencia usando el mejor modelo Time2Vec disponible en el benchmark actual. Ambos paneles se evalúan sobre las mismas coordenadas temporales intermedias, el primero usa las escalas sinusoidales fijas, mientras que el segundo usa los pesos y fases aprendidos por Time2Vec en el checkpoint seleccionado.

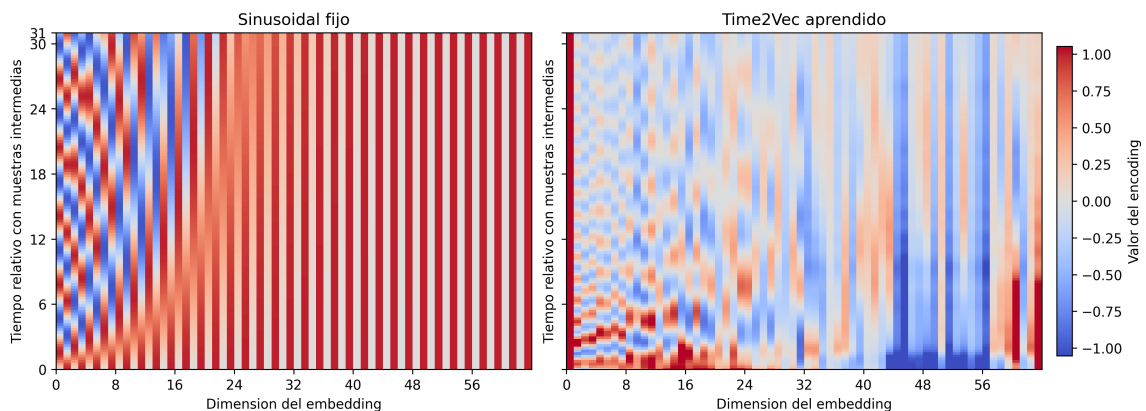


Figura 5.2: Comparación entre un encoding sinusoidal fijo y el encoding Time2Vec aprendido por el mejor modelo Time2Vec disponible en el archivo actual de resultados: **Custom-Time2VecBias-Small**, dataset 4, semilla 126, con MSE de test 0,2094. Ambos paneles usan las mismas coordenadas temporales intermedias y la misma escala de color.

5.3.3. Embedding de flag historia/target

El embedding de rol introduce información explícita sobre la función de cada token en la secuencia. A partir de la máscara historia–objetivo $f \in \{0, 1\}^L$, se construye un *embedding* discreto:

$$f_t = \begin{cases} e_{\text{historia}} \in \mathbb{R}^{d_{\text{model}}}, & \text{si el token es de historia,} \\ e_{\text{target}} \in \mathbb{R}^{d_{\text{model}}}, & \text{si el token es el objetivo.} \end{cases}$$

En la práctica, se usa una tabla aprendible de tamaño 2:

- índice 0 $\rightarrow$ token de historia,
- índice 1 $\rightarrow$ token objetivo.

Esto produce un tensor $e_t^{(\text{flag})} \in \mathbb{R}^{d_{\text{model}}}$ para cada posición.

Conviene subrayar que este embedding no pretende codificar el horizonte de predicción. Su función es únicamente distinguir entre observaciones ya vistas y posiciones objetivo, la información temporal fina sigue residiendo en el encoding continuo basado en timestamps.

5.3.4. Combinación de embeddings y paso por el encoder

Una vez calculados los tres componentes de representación,

$$e_t^{(\text{val})}, \quad e_t^{(\text{time})}, \quad e_t^{(\text{flag})},$$

el modelo construye, para cada posición temporal o token objetivo, un único vector de entrada mediante suma:

$$z_t = e_t^{(\text{val})} + e_t^{(\text{time})} + e_t^{(\text{flag})},$$

de modo que la secuencia completa entra al encoder como el estado inicial

$$H^{(0)} \in \mathbb{R}^{B \times L \times d_{\text{model}}},$$

donde B es el tamaño del batch y L es la longitud total luego de concatenar historia y tokens objetivo.

La estructura interna del bloque encoder ya fue descrita en el Capítulo 4, en esta sección interesa precisar el formato específico usado por la implementación. Los tokens objetivo se ubican siempre al final de la secuencia, y la máscara de rol permite extraer posteriormente sus estados contextualizados. Cuando existen historias de distinta longitud en el mismo batch, se agregan posiciones de relleno y se entrega al encoder una máscara de padding alineada con $H^{(0)}$, de manera que esas posiciones no participen en la atención.

En la configuración base se utiliza atención completa: los tokens objetivo pueden atender a toda la historia disponible y, si hay varios objetivos en la misma secuencia, también se contextualizan conjuntamente. La máscara causal se reserva para variantes autoregresivas o experimentales, no para los resultados principales del encoder base.

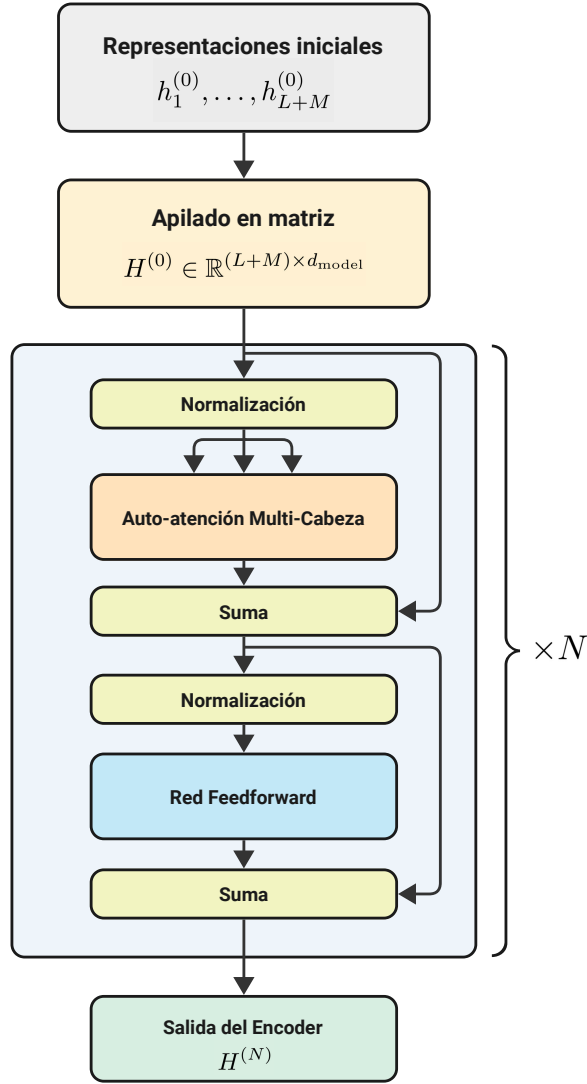


Figura 5.3: Esquema del encoder utilizado en la arquitectura base. Las representaciones iniciales de historia y tokens objetivo se apilan en la matriz $H^{(0)}$ y luego atraviesan N bloques encoder compuestos por normalización, auto-atención multi-cabeza, conexión residual, normalización, red feedforward y una segunda conexión residual.

Formalmente, si se omite la dimensión de batch y se denota por L_{hist} el número de tokens históricos y por M el número de tokens objetivo, la secuencia embebida puede escribirse como:

$$H^{(0)} = \begin{bmatrix} z_1^{(\text{hist})} \\ \vdots \\ z_{L_{\text{hist}}}^{(\text{hist})} \\ z_1^{(\text{target})} \\ \vdots \\ z_M^{(\text{target})} \end{bmatrix} \in \mathbb{R}^{(L_{\text{hist}}+M) \times d_{\text{model}}}.$$

La figura usa esta misma idea de apilamiento: primero aparecen las representaciones

iniciales $h_1^{(0)}, \dots, h_{L+M}^{(0)}$ y luego la matriz $H^{(0)}$ que ingresa a la pila de bloques encoder.

Cada bloque encoder recibe una matriz de estados $H^{(\ell)}$ y produce una nueva matriz $H^{(\ell+1)}$ de la misma dimensión. La implementación utiliza una variante pre-normalizada, es decir, la normalización por capa se aplica antes de cada subcapa. En las ecuaciones siguientes, LN denota normalización por capa (*Layer Normalization*), MHSA denota auto-atención multi-cabeza y FFN denota la red feedforward aplicada posición a posición:

$$U^{(\ell)} = H^{(\ell)} + \text{MHSA}(\text{LN}(H^{(\ell)})), \quad (5.1)$$

$$H^{(\ell+1)} = U^{(\ell)} + \text{FFN}(\text{LN}(U^{(\ell)})), \quad \ell = 0, \dots, N-1. \quad (5.2)$$

Las dos sumas corresponden a las conexiones residuales indicadas en la Figura 5.3. Estas conexiones preservan una ruta directa para la información de entrada y facilitan el entrenamiento de pilas profundas, mientras que las normalizaciones estabilizan la escala de las activaciones antes de la atención y antes de la red feedforward.

En la auto-atención multi-cabeza, las consultas, claves y valores se obtienen desde la misma representación normalizada del encoder:

$$Q = \text{LN}(H^{(\ell)})W^Q, \quad K = \text{LN}(H^{(\ell)})W^K, \quad V = \text{LN}(H^{(\ell)})W^V.$$

Para cada cabeza, la atención calcula pesos entre todos los pares de posiciones válidas:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + \mathcal{M}_{\text{pad}} + \mathcal{B}_{\text{temp}} \right) V.$$

Aquí $\mathcal{M}_{\text{pad}}$ representa la máscara de padding, que anula posiciones agregadas sólo para completar el batch, y $\mathcal{B}_{\text{temp}}$ representa el sesgo temporal opcional descrito en la sección siguiente. Cuando el sesgo temporal no se utiliza, este término se toma igual a cero. Las salidas de las distintas cabezas se concatenan y se proyectan nuevamente a dimensión d_{model} .

La red feedforward se aplica de manera independiente a cada posición:

$$\text{FFN}(u) = W_2 \sigma(W_1 u + b_1) + b_2,$$

donde σ corresponde a la activación configurada para el experimento, ReLU en las configuraciones principales. Esta subcapa no mezcla posiciones entre sí, sino que transforma cada representación contextualizada después de la auto-atención. Tras repetir el bloque N veces se obtiene:

$$H^{(N)} \in \mathbb{R}^{B \times L \times d_{\text{model}}},$$

que contiene una representación contextualizada para cada token histórico y para cada token objetivo.

Sesgo de atención temporal

Dado que el encoding temporal se inyecta mediante suma a los embeddings de entrada, la información temporal llega al mecanismo de atención de forma indirecta. Para reforzar la conciencia temporal dentro de la atención, se introduce un *sesgo de atención temporal* (*temporal attention bias*) que modifica directamente los scores de atención en función de la distancia temporal entre tokens.

Formalmente, para cada cabeza de atención h , los scores modificados son:

$$\text{score}_h(i, j) = \frac{Q_i \cdot K_j^\top}{\sqrt{d_{\text{head}}}} + b_h(\Delta t_{i,j}),$$

donde el sesgo temporal se define como:

$$b_h(\Delta t) = \text{clamp}\left(-\log\left(1 + \frac{|\Delta t|}{\tau_h}\right), -12, 0\right), \quad \tau_h = \exp(\text{clamp}(\alpha_h, -5, 5)),$$

y α_h es un parámetro escalar aprendido por cada cabeza.

La formulación incorpora dos restricciones de estabilidad numérica. Primero, el parámetro α_h se restringe al intervalo $[-5, 5]$, lo que mantiene $\tau_h \in [e^{-5}, e^5] \approx [0,007, 148]$; sin esta restricción, si α_h evoluciona hacia valores muy negativos durante el entrenamiento, $\tau_h \rightarrow 0$ y el cociente $|\Delta t|/\tau_h \rightarrow \infty$, produciendo valores de sesgo que desestabilizan el *softmax*. Segundo, el uso de $\log(1 + \cdot)$ suaviza la penalización para distancias largas y el sesgo resultante se acota al rango $[-12, 0]$, evitando que valores extremos saturen la distribución de atención y propaguen gradientes no finitos.

Este mecanismo tiene tres propiedades deseables:

1. **Localidad temporal aprendida:** cada cabeza aprende su propia escala temporal τ_h . Cabezas con τ_h pequeño se especializan en dependencias de corto plazo, mientras que cabezas con τ_h grande capturan relaciones de largo plazo.
2. **Sensibilidad a la irregularidad:** a diferencia del *positional encoding* ordinal, el sesgo opera sobre diferencias temporales reales ($|t_i - t_j|$), no sobre índices de posición. Dos observaciones a 10 segundos de distancia reciben un sesgo diferente que dos observaciones a 2 horas, independientemente de su posición ordinal en la secuencia.
3. **Complementariedad:** el sesgo se suma a los scores estándar de atención, por lo que no reemplaza sino que complementa la capacidad del modelo de atender por contenido.

Aunque la formulación anterior se desarrolló de manera independiente para el caso de *timestamps* continuos, es consistente con la idea validada por ALiBi Press et al., 2022: incorporar información posicional o temporal como un sesgo aditivo directamente sobre los scores de atención, en vez de depender sólo de un encoding sumado a los embeddings de entrada.

5.3.5. Cabeza de regresión en el token objetivo

Una vez obtenido el tensor de salida del encoder,

$$H^{(N)} \in \mathbb{R}^{B \times L \times d_{\text{model}}},$$

se extraen los estados correspondientes a los tokens objetivo mediante la máscara de rol. Si hay M tokens objetivo por secuencia, esto produce:

$$h_{b,m}^{(\text{target})} \in \mathbb{R}^{d_{\text{model}}}, \quad b = 1, \dots, B, \quad m = 1, \dots, M.$$

La cabeza de regresión aplica sobre cada $h_{b,m}^{(\text{target})}$ una transformación lineal (o una pequeña MLP opcional) para producir la predicción:

$$\hat{y}_{b,m} = W_{\text{out}} h_{b,m}^{(\text{target})} + b_{\text{out}} \in \mathbb{R}^{D_{\text{out}}}, \quad m = 1, \dots, M.$$

En la configuración utilizada en los experimentos, se emplea la versión simple de una sola capa lineal, lo que reduce el número de parámetros y facilita la interpretación de la contribución del encoder.

5.3.6. Variante con resumen atencional de la historia

Además de la lectura directa del token objetivo, el código experimental incorpora una variante que fusiona dicho estado con un resumen atencional de la historia. En esta variante, se calculan pesos aprendidos sobre los tokens históricos válidos y se obtiene un vector global:

$$\bar{h}_b = \sum_{i \in \mathcal{H}_b} \alpha_{b,i} H_{b,i}^{(N)}, \quad \sum_{i \in \mathcal{H}_b} \alpha_{b,i} = 1.$$

Luego, para cada token objetivo, se concatena $h_{b,m}^{(\text{target})}$ con $\bar{h}_b$ y se aplica una pequeña red de fusión antes de la cabeza de regresión. Esta variante permite evaluar si una señal global explícita de la historia mejora la predicción frente a utilizar únicamente el estado contextualizado del token objetivo.

5.3.7. Variante encoder–decoder

Además de las variantes basadas sólo en encoder, se evalúa una formulación encoder–decoder. Su objetivo es separar de manera explícita dos roles que en la arquitectura base conviven dentro de una misma secuencia:

- la **historia observada**, que se procesa en el encoder y produce una memoria contextual,
- los **tokens objetivo**, que se procesan en el decoder, actúan como consultas hacia esa memoria.

Sea L_{hist} la longitud de la historia y L_{obj} la cantidad de tokens objetivo. Primero se construyen embeddings para los tokens históricos usando valores observados, timestamps e identificador de historia. Estos embeddings forman el estado inicial del encoder:

$$H_{\text{hist}}^{(0)} \in \mathbb{R}^{B \times L_{\text{hist}} \times d_{\text{model}}}.$$

Luego, la historia se procesa mediante N_{enc} bloques encoder:

$$H_{\text{hist}}^{(\ell+1)} = \text{EncBlock}^{(\ell)}(H_{\text{hist}}^{(\ell)}), \quad \ell = 0, \dots, N_{\text{enc}} - 1.$$

Cada bloque encoder mantiene la estructura Transformer descrita en el Capítulo 4 y en la Sección 5.3.4: auto-atención multi-cabeza sobre los tokens históricos, conexión residual, normalización por capa y red feedforward. La salida

$$M_{\text{enc}} = H_{\text{hist}}^{(N_{\text{enc}})} \in \mathbb{R}^{B \times L_{\text{hist}} \times d_{\text{model}}}$$

actúa como **memoria histórica**: contiene una representación contextualizada de las observaciones disponibles, pero no incluye tokens objetivo.

En paralelo, se construyen embeddings para los tokens objetivo usando placeholders de valor, sus timestamps futuros y el identificador de objetivo. Esto define la entrada inicial del decoder:

$$D^{(0)} \in \mathbb{R}^{B \times L_{\text{obj}} \times d_{\text{model}}}.$$

El decoder aplica N_{dec} bloques. Cada bloque recibe los estados objetivo $D^{(r)}$ y la memoria histórica M_{enc} :

$$D^{(r+1)} = \text{DecBlock}^{(r)}(D^{(r)}, M_{\text{enc}}), \quad r = 0, \dots, N_{\text{dec}} - 1.$$

Usando la misma notación introducida para el encoder, un bloque decoder pre-normalizado añade una subcapa adicional de atención cruzada. En las ecuaciones siguientes, MHCA denota atención cruzada multi-cabeza:

$$U^{(r)} = D^{(r)} + \text{MHSA}\left(\text{LN}(D^{(r)})\right), \quad (5.3)$$

$$V^{(r)} = U^{(r)} + \text{MHCA}\left(\text{LN}(U^{(r)}), M_{\text{enc}}\right), \quad (5.4)$$

$$D^{(r+1)} = V^{(r)} + \text{FFN}\left(\text{LN}(V^{(r)})\right). \quad (5.5)$$

La auto-atención multi-cabeza mezcla información entre tokens objetivo, opcionalmente puede usar una máscara causal si se desea generación autoregresiva. La atención cruzada multi-cabeza, en cambio, usa los estados del decoder como consultas y la memoria del encoder como claves y valores:

$$Q = \text{LN}(U^{(r)})W_Q, \quad K = M_{\text{enc}}W_K, \quad V = M_{\text{enc}}W_V,$$

$$\text{MHCA}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V,$$

aplicado por cabeza y seguido de concatenación y proyección lineal, como en la atención multi-cabeza estándar. Además, se aplica la máscara de padding de la historia sobre la memoria para impedir que el decoder atienda a posiciones de relleno.

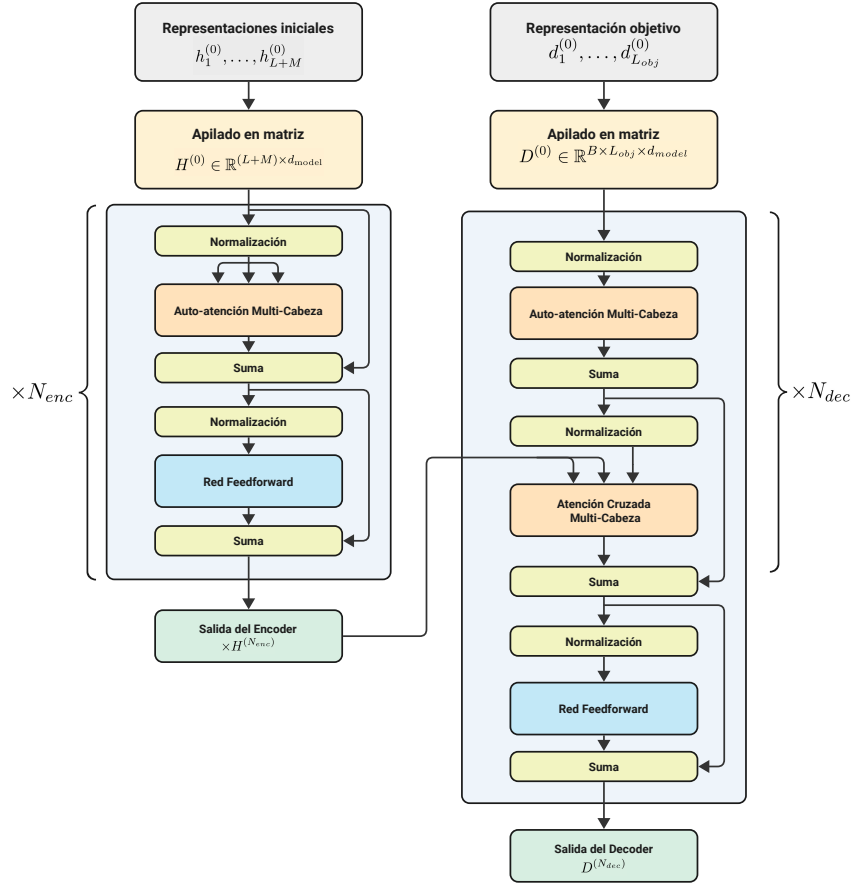


Figura 5.4: Esquema de la variante encoder–decoder. La historia observada se procesa en el encoder para producir una memoria contextual, mientras que los tokens objetivo se procesan en el decoder mediante auto-atención multi-cabeza y atención cruzada multi-cabeza hacia la salida del encoder.

Tras los N_{dec} bloques se obtiene:

$$D^{(N_{\text{dec}})} \in \mathbb{R}^{B \times L_{\text{obj}} \times d_{\text{model}}},$$

y la cabeza de regresión se aplica a cada estado objetivo final:

$$\hat{y}_{b,m} = W_{\text{out}} D_{b,m}^{(N_{\text{dec}})} + b_{\text{out}}, \quad m = 1, \dots, L_{\text{obj}}.$$

En modo no autoregresivo, todos los timestamps objetivo pueden entregarse simultáneamente al decoder. En modo autoregresivo, la memoria del encoder se calcula una vez con la historia observada, y el decoder genera los objetivos secuencialmente: la predicción anterior se usa como entrada de valor del siguiente token objetivo. Esta diferencia es importante: las predicciones previas no se agregan de vuelta a la historia del encoder, sino que se incorporan en el lado del decoder. De este modo, la variante encoder–decoder evalúa si separar memoria histórica y consultas futuras aporta ventajas frente a la arquitectura base de encoder con tokens objetivo.

5.4. Manejo de historias variables y offsets de predicción

Una de las características distintivas de CT-Transformer es la capacidad de entrenar con historias de longitud variable y diferentes *offsets de predicción* dentro del mismo dataset. Esto se logra combinando:

- la lógica de muestreo de historias y offsets,
- la flexibilidad de la etapa de agrupación por lotes para manejar secuencias de tamaños distintos mediante *padding*,
- la invariancia del encoder a longitudes específicas de secuencia.

Desde un punto de vista conceptual, durante el entrenamiento el modelo observa ejemplos de la forma:

$$\left(\{(t_{a-h}, x_{a-h}), \dots, (t_{a-1}, x_{a-1})\}, \{t_m^*\}_{m=1}^M, \{y_m^*\}_{m=1}^M \right),$$

donde h varía entre $h_{\min}$ y $h_{\max}$, y cada t_m^* puede corresponder a distintos índices futuros $a + o_m$. Esto fuerza al modelo a aprender:

- aprovechar tanto historias cortas como largas,
- producir predicciones coherentes para horizontes de predicción heterogéneos.

En los experimentos de este trabajo se analizan explícitamente los efectos de:

1. usar historias fijas vs historias variables,
2. entrenar con un único *offset* vs múltiples *offsets*.

5.5. Función de pérdidas, métricas y bucle de entrenamiento

5.5.1. Función de pérdida

Dado un batch de tamaño B , el modelo produce predicciones

$$\hat{Y} \in \mathbb{R}^{B \times D_{\text{out}}}, \quad Y \in \mathbb{R}^{B \times D_{\text{out}}}$$

son los targets correspondientes. La implementación base utiliza el error cuadrático medio (MSE) como función de pérdida:

$$\mathcal{L}_{\text{MSE}}(\hat{Y}, Y) = \frac{1}{BD_{\text{out}}} \sum_{b=1}^B \sum_{d=1}^{D_{\text{out}}} (\hat{y}_{b,d} - y_{b,d})^2.$$

No obstante, el flujo de entrenamiento permite cambiar fácilmente a MAE ($L1$) o Huber, para estudiar robustez frente a *outliers*.

5.5.2. Métricas de evaluación

Además de la pérdida de entrenamiento, se calcula un conjunto de métricas de regresión a nivel de validación:

- MSE y RMSE,
- MAE,
- MAPE (*Mean Absolute Percentage Error*).

Estas métricas se almacenan por época, lo que permite analizar la evolución del entrenamiento y comparar configuraciones de modelo e hiperparámetros.

5.5.3. Optimización y scheduler

El entrenamiento se gestiona mediante un bucle homogéneo para todos los modelos, que se apoya en:

- un optimizador configurable, con opciones como Adam, AdamW o SGD,
- un *scheduler* de *learning rate*, en el benchmark final se utiliza calentamiento lineal seguido de decaimiento coseno.

En la mayoría de los experimentos se emplea AdamW con regularización de pesos, coherente con la práctica común en modelos Transformer. Adicionalmente, se aplica recorte de gradientes para evitar explosión de gradientes.

La configuración de entrenamiento se carga desde archivos YAML, facilitando la reproducción de experimentos. El detalle del optimizador AdamW y de la política de tasa de aprendizaje se presenta en el Anexo B.

5.6. Inferencia y pronósticos multi-step

Para simplificar el uso del modelo entrenado en escenarios reales, se definió una capa de inferencia de alto nivel:

- **Predicción puntual:** recibe una historia y un timestamp objetivo, y se encarga de:
 - aplicar los *scalers* adecuados a las entradas,
 - construir la secuencia con token objetivo a partir de (historia, t^*),
 - des-escalar las predicciones de salida.
- **Pronóstico rodante:** genera pronósticos multi-step sobre una lista de *timestamps* objetivo, manteniendo fija la historia observada.

Formalmente, dado un conjunto de *timestamps* futuro $\{t_1^*, \dots, t_K^*\}$ y una historia $(t_{i-h+1}, x_{i-h+1}), \dots, (t_i, x_i)$, el pronóstico rodante produce:

$$\hat{Y}_{\text{future}} = \begin{bmatrix} \hat{y}(t_1^*) \\ \vdots \\ \hat{y}(t_K^*) \end{bmatrix} \in \mathbb{R}^{K \times D_{\text{out}}}.$$

Este esquema permite, por ejemplo, construir curvas de pronóstico sobre un horizonte temporal arbitrario, sin necesidad de rehacer manualmente el bucle de construcción de secuencias.

5.7. Comentarios sobre complejidad y limitaciones

Desde el punto de vista computacional, el modelo conserva la complejidad cuadrática típica del Transformer en la longitud de la secuencia:

$$\mathcal{O}(L^2 \cdot d_{\text{model}}),$$

donde L es la longitud efectiva (historia + tokens objetivo). El cálculo del encoding temporal y del sesgo de atención temporal tienen coste lineal y cuadrático en L respectivamente, pero en la práctica ambos son despreciables frente al coste de la auto-atención.

En cuanto al número de parámetros adicionales introducidos por Time2Vec y el sesgo de atención temporal, estos son mínimos:

- Time2Vec añade aproximadamente $4 \cdot d_{\text{model}}$ parámetros cuando se incluye la normalización de capa,
- el sesgo de atención temporal añade H parámetros (uno por cabeza de atención),

lo que representa una fracción insignificante del total de parámetros del modelo.

En cuanto a limitaciones, cabe destacar:

- La complejidad cuadrática puede volverse prohibitiva para historias muy largas. En esta memoria se trabaja con historias de tamaño moderado, pero una línea de trabajo evidente consiste en explorar variantes de atención local o dispersa.
- El modelo no requiere imputar una grilla temporal completa: si no existe una medición en un instante, ese instante puede omitirse y el hueco queda representado por los timestamps reales de las observaciones disponibles. En la representación basada en eventos, además, una medición faltante de una variable simplemente no genera token, y los targets faltantes pueden excluirse de la pérdida mediante una máscara. La limitación aparece cuando el hecho de que una medición falte sea informativo en sí mismo, por ejemplo ante fallas de sensores asociadas a ciertas condiciones, mediciones clínicas solicitadas u omitidas según el estado del paciente, o protocolos donde una visita programada sin valor observado comunica información. Incorporar explícitamente esa señal mediante máscaras de observabilidad, tokens especiales de ausencia o imputación integrada se deja como trabajo futuro.

A pesar de estas limitaciones, el diseño de CT-Transformer ofrece una vía concreta y relativamente simple para adaptar los Transformers a series de tiempo no equiespaciadas, manteniendo al mismo tiempo una implementación clara y modular que puede ser extendida en trabajos posteriores.

Capítulo 6

Experimentos

Este capítulo fija la metodología experimental utilizada para evaluar CT-Transformer y sus variantes. La fase experimental de esta memoria se da por cerrada con cinco datasets reales y tres semillas por combinación dataset–modelo. Por lo tanto, el Capítulo 7 reporta conclusiones finales para esta evaluación, aunque el alcance estadístico sigue estando limitado por el número de datasets.

6.1. Objetivos experimentales

Los experimentos persiguen los siguientes objetivos:

1. **Verificar factibilidad:** comprobar que arquitecturas basadas en *Transformers* con timestamps reales y tokens objetivo pueden entrenarse de manera estable sobre series reales no equiespaciadas sin regularizarlas a una grilla temporal fija.
2. **Evaluar desempeño predictivo:** medir MSE, RMSE y MAE en el conjunto de test retenido.
3. **Comparar contra baselines informados:** contrastar la propuesta con STraTS Tipirneni y Reddy, 2022 y CoFormer Wei et al., 2023, adaptados a la tarea de pronóstico multi-horizonte.
4. **Comparar contra baselines simples:** incluir persistencia y un modelo lineal temporal para verificar que las mejoras no provengan sólo de aumentar la complejidad.
5. **Cuantificar ablaciones:** medir el aporte del encoding temporal continuo y del identificador de token objetivo.
6. **Explorar variantes arquitectónicas:** evaluar tamaños pequeños y base, resumen atencional de la historia, Time2Vec, sesgo temporal de atención y variantes encoder–decoder.
7. **Evaluar costo versus beneficio:** registrar parámetros, tiempo de entrenamiento y épocas ejecutadas.
8. **Delimitar el alcance de las conclusiones:** separar las señales consistentes del benchmark cerrado de las afirmaciones que requerirían más datasets, más semillas o evaluación multivariada.

6.2. Escenario experimental

La evaluación final se realiza sobre los archivos `real_data_i.csv`, con $i = 1, \dots, 5$. Cada dataset se ordena temporalmente y se divide respetando el tiempo en entrenamiento, validación y prueba, con proporciones 70 %, 15 % y 15 %, respectivamente. Para cada modelo se ejecutan tres semillas: 42, 84 y 126.

El archivo principal de resultados es `benchmark_final.csv`, ubicado en el directorio de experimentos finales. El subconjunto usado para el análisis final contiene 360 corridas balanceadas: 24 modelos con 15 corridas cada uno, correspondientes a 5 datasets y 3 semillas. Las filas parciales asociadas a otros datasets o variantes incompletas no se usan para extraer conclusiones finales.

El benchmark se ejecuta mediante un orquestador reanudable. Después de cada corrida, las métricas se guardan en un archivo CSV; si el proceso se interrumpe, puede continuar desde el último punto completado. Esta decisión fue importante porque algunos modelos, en especial CoFormer y las variantes encoder–decoder con ajuste autoregresivo, tienen costos de entrenamiento sustanciales.

6.2.1. Variables, partición temporal y preprocesamiento

En la configuración activa de datos reales, cada serie contiene:

- una columna temporal `timestamp`,
- una variable observada `valor`, usada como entrada y como objetivo de predicción,
- una partición temporal entrenamiento–validación–prueba de 70 %–15 %–15 %.

El escalado se ajusta sólo con el segmento de entrenamiento y luego se aplica a validación y prueba, evitando fuga de información desde el futuro. Las ventanas supervisadas usan una historia máxima de 500 observaciones y una historia mínima de 20 observaciones. Para los modelos no autoregresivos se seleccionan hasta cuatro timestamps objetivo dentro de offsets entre 1 y 30 pasos futuros. La escala temporal del modelo se estima por dataset a partir de la mediana de los deltas temporales positivos del bloque de entrenamiento, con límites conservadores para evitar escalas patológicas.

6.3. Modelos comparados

Los modelos evaluados se agrupan en cinco familias.

6.3.1. Familia CT-Transformer

- **CT-Transformer base:** encoder *Transformer* con embedding de valores, encoding temporal sinusoidal continuo, identificador historia–objetivo y cabeza de regresión sobre los tokens objetivo.
- **CT-Transformer pequeño:** misma arquitectura, con menor dimensión interna y menos capas.

- **CT-Transformer con resumen atencional:** fusiona el estado del token objetivo con un resumen aprendido de la historia.
- **CT-Transformer con Time2Vec:** reemplaza el encoding sinusoidal fijo por una representación temporal aprendible basada en Time2Vec Kazemi et al., 2019.
- **CT-Transformer con sesgo temporal:** conserva el encoding sinusoidal, pero añade un sesgo temporal directamente en los scores de atención.
- **CT-Transformer con Time2Vec y sesgo temporal:** combina representación temporal aprendida y sesgo temporal de atención.

6.3.2. Variantes encoder–decoder

Además del encoder único, se evalúan variantes encoder–decoder donde la historia se procesa en el encoder y los timestamps objetivo se procesan en el decoder mediante atención cruzada. La versión optimizada usa un decoder liviano de una capa. Se consideran dos tamaños: pequeño y base.

Las variantes `EncDec-Opt-Small-MT8` y `EncDec-Opt-MT8` se entrenan con ocho tokens objetivo. Después se aplican ajustes autoregresivos con máscara causal y tres modos de provisión de objetivos:

- **Contiguous:** offsets futuros consecutivos.
- **Random:** offsets futuros muestreados desde el rango permitido.
- **Mixed:** mezcla entre entrenamiento contiguo y aleatorio.

En las variantes autoregresivas, la evaluación registrada usa generación autoregresiva sobre ocho objetivos contiguos. Por esta razón, cuando se compara contra modelos no autoregresivos, la lectura correcta es que se compara el desempeño de los sistemas tal como fueron evaluados en el benchmark, no una demostración aislada bajo exactamente el mismo conjunto de consultas futuras.

6.3.3. Baselines del estado del arte

STraTS y CoFormer se incluyen porque son dos referencias representativas para series irregulares sin imputación a una malla fija. No obstante, ambos fueron propuestos principalmente como codificadores de observaciones irregulares para tareas de clasificación, por lo que en este benchmark se usan adaptaciones orientadas a pronóstico multi-horizonte.

- **STraTS adaptado:** conserva la representación por tripletas tiempo–variable–valor y el encoder con fusión por atención de STraTS. Para hacerlo comparable con la tarea de esta memoria, se añaden consultas futuras marcadas como objetivos, de modo que el modelo pueda entregar una predicción por timestamp objetivo. Esta adaptación no convierte a STraTS en un encoder–decoder: sigue siendo un codificador de tripletas con lectura sobre representaciones contextualizadas.

- **CoFormer adaptado:** conserva la idea de puntos variable–tiempo y la separación entre atención intra-variable e inter-variable. En la configuración actual de datos reales, las corridas consolidadas son univariadas, por ello CoFormer opera como **CoFormer-Uni**, donde la atención inter-variable deja de aportar el mecanismo central para el que fue diseñado. Para pronóstico, la representación agregada de la historia se combina con la información temporal de cada objetivo futuro.

6.3.4. Baselines simples

- **Persistencia:** predice el último valor observado de la historia para todos los horizontes futuros.
- **Modelo lineal temporal:** usa los últimos 50 valores de la historia junto con tiempos relativos para predecir cada horizonte.

6.3.5. Ablaciones

- **Sin encoding temporal continuo:** reemplaza los timestamps reales por un encoding posicional ordinal. Mide el aporte específico del tiempo continuo.
- **Sin identificador de token objetivo:** elimina la señal aprendible que distingue historia y objetivo. Mide el aporte de explicitar el rol del token.

6.4. Configuración de entrenamiento

Las configuraciones paramétricas principales son:

- **Pequeña:** $d_{\text{model}} = 64$, 2 capas, 4 cabezas, *feed-forward* 128, *dropout* 0.1.
- **Base:** $d_{\text{model}} = 128$, 4 capas, 4 cabezas, *feed-forward* 512, *dropout* 0.15.

El entrenamiento usa AdamW, calentamiento lineal seguido de decaimiento coseno, selección del mejor checkpoint según RMSE de validación, precisión mixta automática y recorte de gradientes. La configuración se ajusta según el tamaño del modelo. En las variantes más inestables de CT-Transformer y del encoder–decoder optimizado se usa pérdida Huber, menor tasa de aprendizaje, más calentamiento y recorte de gradiente más estricto. El scheduler se detalla en el Anexo B.

6.5. Métricas de evaluación

La evaluación se realiza mediante:

- **MSE:** métrica principal, sensible a errores grandes.
- **RMSE:** raíz del MSE, expresada en la misma escala de la variable normalizada.
- **MAE:** error absoluto medio, menos sensible a valores atípicos.
- **MAPE:** error porcentual absoluto medio, se reporta con cautela porque puede ser inestable cuando el valor real está cerca de cero.

El análisis principal prioriza MSE, RMSE y MAE.

6.6. Agregación y análisis estadístico

Para cada modelo, las tres semillas se promedian primero dentro de cada dataset. Luego se reporta la media y desviación estándar sobre los cinco promedios por dataset. Esta decisión evita tratar semillas como si fueran datasets independientes y permite que cada serie tenga el mismo peso en las tablas finales.

También se reportan victorias por dataset, calculadas con MSE luego de promediar semillas. Dado que sólo hay cinco datasets, las pruebas estadísticas emparejadas tienen baja potencia. Por ello, las conclusiones del Capítulo 7 se basan en magnitud, consistencia entre datasets y comparación de costos, no en una afirmación de significancia estadística fuerte.

6.7. Costo computacional

Para cada corrida se registra el número de parámetros entrenables, el número total de parámetros, el tiempo de entrenamiento en segundos y el número de épocas ejecutadas antes de detener el entrenamiento. En las variantes con ajuste autoregresivo, el costo reportado en las tablas finales corresponde al entrenamiento base más el ajuste posterior, porque ambos pasos son necesarios para obtener el modelo evaluado.

6.8. Reproducibilidad

El benchmark se puede reanudar ejecutando:

```
python scripts/benchmark_final.py
```

El orquestador admite seleccionar rangos de datasets, semillas, subconjuntos de modelos y variantes opcionales como encoder-decoder y ajuste autoregresivo. Para reproducir el análisis final de esta memoria se debe filtrar el archivo de resultados a los datasets 1–5, semillas 42, 84 y 126, y modelos con las 15 corridas completas.

6.9. Resumen metodológico

En síntesis, el diseño experimental compara la familia CT-Transformer, variantes encoder-decoder, baselines simples y adaptaciones de STraTS y CoFormer bajo un protocolo temporalmente justo: entrenamiento en pasado, validación intermedia y prueba futura. La fase experimental cerrada contiene cinco datasets, tres semillas por dataset y 24 modelos completos. El Capítulo 7 utiliza ese subconjunto balanceado para declarar el mejor modelo observado y extraer conclusiones con el nivel de cautela que corresponde a una evaluación de cinco series reales.

Capítulo 7

Resultados y discusión

Este capítulo presenta los resultados finales de la fase experimental definida en el Capítulo 6. El análisis usa el subconjunto balanceado del archivo `benchmark_final.csv`: datasets 1–5, semillas 42, 84 y 126, y sólo modelos con las 15 corridas completas. En total, el subconjunto final contiene 360 corridas, correspondientes a 24 modelos.

Las tablas reportan la media y el intervalo de confianza de la distribución t de Student al 95 % sobre 15 corridas individuales ($5 \text{ datasets} \times 3 \text{ semillas}$), tratándolas como observaciones independientes para aumentar la potencia estadística de los análisis. Valores menores indican mejor desempeño. La métrica principal es MSE; RMSE y MAE se usan como métricas complementarias.

7.1. Cobertura final del benchmark

La Tabla 7.1 resume la cobertura utilizada para el análisis final. El archivo de resultados contiene algunas filas adicionales de corridas parciales, pero no se usan para las conclusiones porque no forman parte del conjunto balanceado.

Tabla 7.1: Cobertura final usada en el análisis.

Grupo	Modelos completos	Corridas completas
Variantes CT-Transformer encoder	10	150
Variantes encoder-decoder optimizadas	8	120
Baselines de estado del arte adaptados	2	30
Baselines simples	2	30
Ablaciones	2	30
Total	24	360

7.2. Metodología de presentación y análisis de resultados

Para asegurar la validez científica y el rigor de las comparaciones presentadas en este capítulo, se adoptan las siguientes definiciones y metodologías estadísticas:

- **Intervalos de Confianza t-Student:** En lugar del formato tradicional de media más/menos desviación estándar (que solo describe la dispersión muestral), cada métrica principal se reporta mediante su media y su intervalo de confianza al 95 % para la media poblacional. Este se calcula utilizando la distribución t de Student con $n - 1$ grados de libertad:

$$\bar{x} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}$$

donde $\bar{x}$ es la media muestral, s es la desviación estándar muestral, $n = 15$ es el número de corridas independientes ($5 \text{ datasets} \times 3 \text{ semillas}$) y $t_{\alpha/2, 14} \approx 2.145$ es el valor crítico para $\alpha = 0.05$ bilateral. Este intervalo define el rango en el cual se sitúa el verdadero error medio esperado del modelo con un 95 % de certeza.

- **Pruebas de Significancia Estadística Emparejadas:** Dado que todos los modelos se evalúan exactamente sobre el mismo conjunto de datasets y semillas, los resultados de desempeño están emparejados. Por ello, para contrastar si la superioridad del modelo ganador es estadísticamente significativa frente a las alternativas, se aplican dos pruebas bilaterales:

1. *Prueba t de Student para muestras emparejadas:* Evalúa si la diferencia media de los errores entre dos modelos es significativamente distinta de cero, asumiendo normalidad en las diferencias de error.
2. *Prueba de rangos con signo de Wilcoxon:* Alternativa no paramétrica que no asume normalidad, basada en el orden de los rangos de las diferencias.

Un p-value (p) menor a 0.05 se considera suficiente para rechazar la hipótesis nula (H_0) de igualdad de rendimiento, declarando una ventaja estadísticamente significativa.

- **Métricas de Error:** Se reportan tres métricas complementarias para evaluar distintas propiedades de las predicciones:
 1. *MSE (Mean Squared Error):* Métrica principal de optimización. Al elevar los errores al cuadrado, penaliza de forma muy estricta las desviaciones grandes, lo cual es crítico en aplicaciones reales donde los errores extremos son costosos.
 2. *RMSE (Root Mean Squared Error):* Mantiene las propiedades del MSE pero escala el error a la misma unidad física de la variable objetivo.
 3. *MAE (Mean Absolute Error):* Promedio de los errores absolutos. Es más robusto frente a valores atípicos y describe el error promedio típico.
- **Victorias:** Indica la proporción de datasets (en este caso, de los 5 datasets balanceados) en los cuales el modelo respectivo obtuvo el menor MSE promedio.

7.3. Resultado principal

El mejor desempeño observado en la evaluación final corresponde a **EncDec-Opt-Small-MT8_FT_AR_Contiguous**. Esta variante obtiene el menor MSE, RMSE y MAE, y gana en los 5 de 5 datasets al comparar promedios por dataset.

Arquitectónicamente, el modelo ganador es un encoder-decoder pequeño: $d_{\text{model}} = 64$, 2 capas de encoder, 4 cabezas de atención, *feed-forward* de dimensión 128 y *dropout* 0.1.

A diferencia del encoder puro CT-Transformer, procesa la historia en un encoder y los tokens objetivo en un decoder liviano de una capa con atención cruzada hacia la historia. La sigla MT8 indica que se entrenó con ocho tokens objetivo. El sufijo FT_AR_Contiguous indica un ajuste posterior autoregresivo con máscara causal y objetivos futuros contiguos.

La Tabla 7.2 compara el modelo ganador con las variantes y baselines más relevantes. El tiempo reportado para modelos con ajuste autoregresivo incluye tanto el entrenamiento base como el ajuste posterior.

Tabla 7.2: Resultados finales de modelos representativos.

Modelo	MSE	RMSE	MAE	Parámetros	Tiempo medio	Victorias
EncDec-Opt pequeño MT8 + AR contiguo	0.1979 [0.1702, 0.2257]	0.4417 [0.4110, 0.4724]	0.2193 [0.1949, 0.2437]	117.8K	0.46 h	5/5
CT-Transformer pequeño + Time2Vec + sesgo temporal	0.2807 [0.2443, 0.3170]	0.5265 [0.4927, 0.5603]	0.2906 [0.2519, 0.3293]	67.8K	0.74 h	0/5
CT-Transformer pequeño	0.2817 [0.2456, 0.3178]	0.5275 [0.4940, 0.5611]	0.2935 [0.2526, 0.3343]	67.4K	0.69 h	0/5
CT-Transformer base	0.2906 [0.2568, 0.3244]	0.5363 [0.5054, 0.5673]	0.2859 [0.2471, 0.3247]	636.2K	0.92 h	0/5
Modelo lineal temporal	0.2919 [0.2558, 0.3281]	0.5372 [0.5043, 0.5702]	0.3286 [0.2871, 0.3700]	10.8K	0.69 h	0/5
STraTS adaptado	0.3274 [0.2765, 0.3783]	0.5668 [0.5219, 0.6118]	0.3205 [0.2763, 0.3647]	229.2K	0.63 h	0/5
Persistencia	0.4201 [0.3144, 0.5258]	0.6365 [0.5664, 0.7066]	0.3435 [0.2530, 0.4341]	0 efectivos	0.00 h	0/5
CoFormer univariado	0.5663 [0.5198, 0.6127]	0.7506 [0.7200, 0.7812]	0.4464 [0.3883, 0.5046]	693.4K	0.22 h	0/5

La reducción de MSE del modelo ganador es de 29.7 % frente a CT-Transformer pequeño, 31.9 % frente a CT-Transformer base, 32.2 % frente al modelo lineal temporal, 39.5 % frente a STraTS adaptado y 52.9 % frente a persistencia. Además, la ventaja no se concentra en un solo dataset: el modelo ganador obtiene el menor MSE promedio en cada uno de los cinco datasets.

7.4. Comparación contra baselines

La comparación contra persistencia muestra que todos los modelos neuronales principales aprenden más que repetir el último valor observado. El resultado más fuerte lo entrega el encoder-decoder autoregresivo: reduce el MSE medio desde 0.4201 hasta 0.1979.

La comparación contra el modelo lineal temporal es más informativa. El lineal alcanza MSE 0.2919, muy cerca de CT-Transformer base y de algunas variantes encoder puras. Esto confirma que el benchmark no es trivial: una parametrización simple con historia reciente y tiempos relativos ya captura parte importante de la dinámica. Aun así, el modelo ganador queda claramente por debajo de ese baseline.

STraTS adaptado obtiene MSE 0.3274, competitivo pero por debajo de las mejores variantes CT-Transformer. CoFormer univariado queda rezagado con MSE 0.5663. Esta diferencia debe interpretarse con cuidado: CoFormer fue diseñado para explotar relaciones entre múltiples variables asincrónicas, mientras que la evaluación final consolidada es univariada. En este contexto pierde buena parte de su sesgo inductivo original.

7.5. Variantes encoder-decoder

La Tabla 7.3 muestra que el ajuste autoregresivo es el cambio arquitectónico más decisivo de la evaluación. La variante encoder-decoder pequeña sin ajuste autoregresivo obtiene MSE 0.2945, similar a CT-Transformer base. Después del ajuste autoregresivo contiguo, el MSE baja a 0.1979, una reducción aproximada de 33 %.

Tabla 7.3: Resultados finales de variantes encoder-decoder optimizadas.

Modelo	MSE	RMSE	MAE	Parámetros	Tiempo medio
EncDec-Opt-Small-MT8	0.2945 [0.2575, 0.3314]	0.5395 [0.5060, 0.5730]	0.2859 [0.2491, 0.3226]	117.8K	0.30 h
EncDec-Opt-Small-MT8 + AR contiguo	0.1979 [0.1702, 0.2257]	0.4417 [0.4110, 0.4724]	0.2193 [0.1949, 0.2437]	117.8K	0.46 h
EncDec-Opt-Small-MT8 + AR mixto	0.2043 [0.1767, 0.2320]	0.4489 [0.4183, 0.4795]	0.2248 [0.1994, 0.2501]	117.8K	0.47 h
EncDec-Opt-Small-MT8 + AR aleatorio	0.2206 [0.1885, 0.2527]	0.4659 [0.4317, 0.5001]	0.2369 [0.2075, 0.2664]	117.8K	0.47 h
EncDec-Opt-MT8	0.3028 [0.2642, 0.3413]	0.5470 [0.5125, 0.5814]	0.2909 [0.2528, 0.3289]	1.06M	0.53 h
EncDec-Opt-MT8 + AR contiguo	0.2102 [0.1775, 0.2429]	0.4544 [0.4195, 0.4893]	0.2240 [0.1978, 0.2502]	1.06M	0.76 h
EncDec-Opt-MT8 + AR mixto	0.2172 [0.1833, 0.2512]	0.4619 [0.4260, 0.4977]	0.2285 [0.1997, 0.2572]	1.06M	0.79 h
EncDec-Opt-MT8 + AR aleatorio	0.2424 [0.1993, 0.2855]	0.4866 [0.4437, 0.5295]	0.2430 [0.2095, 0.2764]	1.06M	0.79 h

El patrón es consistente: las versiones autoregresivas superan a sus respectivas bases, el modo contiguo es el mejor entre los tres modos de ajuste y el tamaño pequeño domina al tamaño base. Esto sugiere que, para estos cinco datasets univariados, el beneficio principal no proviene de aumentar capacidad, sino de alinear el entrenamiento con la generación secuencial de horizontes futuros.

Hay una cautela metodológica importante. Las variantes autoregresivas se evalúan con generación sobre ocho objetivos contiguos, mientras que los modelos no autoregresivos se evalúan con hasta cuatro objetivos dentro del rango de offsets configurado. Por ello, el resultado permite declarar al encoder-decoder pequeño con AR contiguo como el mejor sistema observado en esta evaluación, pero no aísla completamente si la mejora proviene del decoder, del ajuste autoregresivo, del protocolo de objetivos o de la combinación de estos factores.

7.6. Variantes CT-Transformer

Dentro de las variantes encoder puras de CT-Transformer, la señal principal es que los modelos pequeños son suficientes y, en MSE, levemente mejores que las versiones base. La Tabla 7.4 resume las variantes más relevantes.

Tabla 7.4: Resultados finales de variantes encoder de CT-Transformer.

Modelo	MSE	RMSE	MAE	Parámetros
CT-Transformer pequeño + Time2Vec + sesgo temporal	0.2807 [0.2443, 0.3170]	0.5265 [0.4927, 0.5603]	0.2906 [0.2519, 0.3293]	67.8K
CT-Transformer pequeño + sesgo temporal	0.2808 [0.2447, 0.3168]	0.5266 [0.4931, 0.5602]	0.2915 [0.2529, 0.3300]	67.6K
CT-Transformer pequeño + resumen atencional	0.2817 [0.2455, 0.3178]	0.5275 [0.4940, 0.5610]	0.2934 [0.2541, 0.3326]	82.1K
CT-Transformer pequeño	0.2817 [0.2456, 0.3178]	0.5275 [0.4940, 0.5611]	0.2935 [0.2526, 0.3343]	67.4K
CT-Transformer pequeño + Time2Vec	0.2824 [0.2463, 0.3184]	0.5282 [0.4948, 0.5616]	0.2938 [0.2540, 0.3335]	67.8K
CT-Transformer base + Time2Vec + sesgo temporal	0.2851 [0.2498, 0.3203]	0.5309 [0.4984, 0.5634]	0.2756 [0.2395, 0.3116]	794.9K
CT-Transformer base + sesgo temporal	0.2846 [0.2483, 0.3208]	0.5302 [0.4969, 0.5636]	0.2777 [0.2411, 0.3143]	794.4K
CT-Transformer base + Time2Vec	0.2867 [0.2505, 0.3228]	0.5323 [0.4990, 0.5655]	0.2773 [0.2381, 0.3165]	794.9K
CT-Transformer base + resumen atencional	0.2869 [0.2506, 0.3231]	0.5324 [0.4991, 0.5658]	0.2799 [0.2406, 0.3193]	852.1K
CT-Transformer base	0.2906 [0.2568, 0.3244]	0.5363 [0.5054, 0.5673]	0.2859 [0.2471, 0.3247]	636.2K

La mejor variante encoder pura por MSE es CT-Transformer pequeño con Time2Vec y sesgo temporal, pero su ventaja frente a CT-Transformer pequeño es sólo de 0.4 % en MSE. Por lo tanto, la conclusión razonable no es que Time2Vec más sesgo temporal sea decisivo, sino que la familia pequeña de CT-Transformer es robusta y difícil de distinguir con sólo cinco datasets.

En MAE, las variantes base con Time2Vec o sesgo temporal mejoran al modelo pequeño. Esto sugiere que mayor capacidad y representaciones temporales más flexibles pueden reducir errores absolutos típicos, aunque no necesariamente los errores cuadráticos grandes que dominan el MSE.

7.7. Ablaciones

La Tabla 7.5 muestra las dos ablaciones principales. La eliminación del identificador historia-objetivo degrada el desempeño: el MSE sube desde 0.2906 hasta 0.3028, una pérdida cercana a 4.0 %. Esta señal respalda la decisión de marcar explícitamente qué tokens son observaciones pasadas y cuáles son consultas futuras.

Tabla 7.5: Ablaciones finales respecto de CT-Transformer base.

Modelo	MSE	RMSE	MAE	Parámetros
CT-Transformer base	0.2906 [0.2568, 0.3244]	0.5363 [0.5054, 0.5673]	0.2859 [0.2471, 0.3247]	636.2K
Sin encoding temporal continuo	0.2903 [0.2534, 0.3271]	0.5356 [0.5021, 0.5691]	0.2957 [0.2579, 0.3335]	583.4K
Sin identificador de token objetivo	0.3028 [0.2698, 0.3357]	0.5478 [0.5177, 0.5778]	0.3027 [0.2680, 0.3374]	600.7K
CT-Transformer pequeño	0.2817 [0.2456, 0.3178]	0.5275 [0.4940, 0.5611]	0.2935 [0.2526, 0.3343]	67.4K

El efecto del encoding temporal continuo es menos claro. La ablación sin tiempo continuo queda prácticamente empatada con CT-Transformer base en MSE y RMSE, aunque empeora en MAE. En estos cinco datasets univariados, no hay evidencia suficiente para afirmar que el encoding temporal continuo sea por sí solo el componente decisivo. La evidencia sí favorece el uso del identificador de token objetivo.

La comparación con CT-Transformer pequeño también muestra que reducir capacidad mejora el MSE frente al modelo base. Esto sugiere que las series evaluadas no requieren necesariamente un encoder de mayor tamaño, o que la regularización implícita de la versión pequeña resulta favorable.

7.8. Costo computacional

El resultado de costo es favorable para el modelo ganador. Aunque requiere dos etapas, entrenamiento base y ajuste autoregresivo, su tiempo medio total es 0.46 horas por corrida y usa 117.8K parámetros. Es más costoso que persistencia, pero más barato que CT-Transformer pequeño en el registro final y bastante más pequeño que CT-Transformer base.

El aumento de tamaño no entrega una mejora clara. CT-Transformer base usa 636.2K parámetros y obtiene MSE 0.2906, mientras que CT-Transformer pequeño usa 67.4K y obtiene MSE 0.2817. Algo similar ocurre en encoder-decoder: la variante pequeña con ajuste AR contiguo supera a la variante base con el mismo ajuste, pese a tener aproximadamente una novena parte de los parámetros.

7.9. Significancia estadística y alcance

Para robustecer las conclusiones sobre el desempeño relativo de los modelos y mitigar la limitación del tamaño de muestra al promediar semillas, se trataron las 15 corridas (5 datasets $\times$ 3 semillas) como observaciones independientes emparejadas por par (dataset, semilla). Sobre estas 15 corridas independientes se aplicaron dos pruebas estadísticas bilaterales para evaluar la significancia de la diferencia en MSE: la prueba t de Student para muestras emparejadas y la prueba no paramétrica de rangos con signo de Wilcoxon.

Al comparar el modelo ganador, `EncDec-Opt-Small-MT8_FT_AR_Contiguous`, contra los baselines y variantes clave en la Tabla 7.2, los resultados son concluyentes:

- **Frente a CT-Transformer base:** La reducción de error es altamente significativa tanto en la prueba t de Student ($t = -20.55$, $p < 0.0001$) como en la prueba de Wilcoxon ($W = 0.0$, $p \approx 0.00006$).
- **Frente a la mejor variante de encoder (CT-Transformer pequeño + Time2Vec + sesgo temporal):** La ventaja del modelo encoder-decoder es también muy robusta estadísticamente ($t = -16.42$, $p < 0.0001$; Wilcoxon $W = 0.0$, $p \approx 0.00006$).
- **Frente al Modelo lineal temporal:** Se observa significancia estadística rigurosa ($t = -18.28$, $p < 0.0001$; Wilcoxon $W = 0.0$, $p \approx 0.00006$).
- **Frente a STraTS adaptado y Persistencia:** Las diferencias de medias favorecen al modelo ganador con un nivel de confianza superior al 99.9 % ($p < 0.0001$ en la prueba t de Student, y $W = 0.0$, $p \approx 0.00006$ en Wilcoxon).

Estos resultados indican que la superioridad del modelo ganador no es producto de fluctuaciones aleatorias en las semillas, sino que existe una diferencia de desempeño sistemática y estadísticamente significativa frente a todas las demás arquitecturas evaluadas bajo los mismos datasets y condiciones de entrenamiento.

7.10. Discusión sobre MAPE

El MAPE presenta alta variabilidad en las corridas actuales y, en algunas variantes autoregresivas, se registra en columnas auxiliares separadas. Esto es esperable cuando la variable objetivo puede acercarse a cero, porque pequeños denominadores inflan artificialmente el error porcentual. Por ello, el análisis principal se basa en MSE, RMSE y MAE.

7.11. Síntesis del capítulo

Los resultados finales permiten resumir la evaluación en cinco puntos:

1. El mejor modelo observado es `EncDec-Opt-Small-MT8_FT_AR_Contiguous`: encoder-decoder pequeño, entrenado con ocho objetivos y ajustado autoregresivamente sobre objetivos contiguos.
2. Este modelo gana en los 5 datasets y obtiene MSE 0.1979, RMSE 0.4417 y MAE 0.2193.
3. Dentro de los encoders puros, las variantes pequeñas de CT-Transformer son las más competitivas; la mejor por MSE es la versión pequeña con Time2Vec y sesgo temporal, aunque queda prácticamente empatada con CT-Transformer pequeño.
4. Las ablaciones indican que el identificador de token objetivo aporta una señal útil; el efecto aislado del encoding temporal continuo no queda demostrado en este subconjunto.

5. Aumentar capacidad no mejora el MSE en esta evaluación. Los modelos pequeños entregan la mejor relación entre error, parámetros y tiempo.

Capítulo 8

Conclusiones y trabajo futuro

La pregunta central de esta memoria es si es posible adaptar arquitecturas *Transformer* para trabajar directamente con series de tiempo no equiespaciadas, incorporando timestamps reales sin regularizar previamente la serie sobre una malla temporal fija. La respuesta, dentro del alcance de esta evaluación experimental, es afirmativa.

El trabajo implementó un pipeline completo para construir ventanas irregulares, añadir tokens objetivo, entrenar modelos con distintas arquitecturas, evaluar múltiples horizontes futuros y comparar contra baselines simples y modelos adaptados del estado del arte. La fase experimental se cerró con cinco datasets reales, tres semillas por dataset y 24 modelos completos. Aunque este tamaño muestral no permite afirmar significancia estadística fuerte, sí permite declarar tendencias y seleccionar el mejor modelo observado.

8.1. Conclusiones principales

Las conclusiones principales son:

1. El mejor modelo de la evaluación final es el encoder–decoder pequeño MT8 con AR contiguo. Este modelo obtiene MSE 0.1979, RMSE 0.4417 y MAE 0.2193, y gana en los cinco datasets al comparar promedios por dataset.
2. La mejor arquitectura no fue el encoder base, sino una extensión encoder–decoder autoregresiva. El modelo ganador usa una configuración pequeña, con $d_{\text{model}} = 64$, 2 capas de encoder, 4 cabezas de atención, *feed-forward* 128, *dropout* 0.1, decoder de una capa, entrenamiento con ocho tokens objetivo y ajuste autoregresivo sobre objetivos contiguos.
3. El ajuste autoregresivo fue el factor arquitectónico más fuerte. La variante encoder–decoder pequeña sin ajuste obtiene MSE 0.2945; con ajuste AR contiguo baja a 0.1979. La reducción cercana a 33 % sugiere que alinear entrenamiento y generación secuencial de horizontes futuros es especialmente beneficioso en estos datos.
4. Dentro de los encoders puros, las variantes pequeñas de CT-Transformer son las más competitivas. La mejor variante encoder por MSE es CT-Transformer pequeño con Time2Vec y sesgo temporal, con MSE 0.2807, pero queda prácticamente empatada con CT-Transformer pequeño, que obtiene 0.2817.

5. Aumentar capacidad no mejoró el MSE. CT-Transformer base y el encoder–decoder base tienen más parámetros que sus versiones pequeñas, pero no superan su desempeño. En esta evaluación, los modelos pequeños entregan la mejor relación entre error, tamaño y tiempo.
6. El identificador de token objetivo aporta una señal útil. Al eliminarlo, el MSE de CT-Transformer base sube desde 0.2906 hasta 0.3028. Esto respalda la decisión de distinguir explícitamente observaciones históricas y consultas futuras.
7. El aporte aislado del encoding temporal continuo no queda demostrado. La ablación sin encoding temporal continuo empata en MSE con CT-Transformer base, aunque empeora en MAE. Por lo tanto, la evidencia favorece la formulación completa, pero no permite afirmar que el encoding temporal sinusoidal sea por sí solo el componente decisivo.
8. Los baselines simples siguen siendo importantes. El modelo lineal temporal alcanza MSE 0.2919, muy cercano al encoder base. Esto muestra que el benchmark exige comparaciones cuidadosas: parte de la estructura predictiva puede capturarse con modelos simples.
9. STraTS adaptado es competitivo, mientras que CoFormer univariado queda rezagado. STraTS obtiene MSE 0.3274. CoFormer univariado obtiene 0.5663, resultado esperable en parte porque su diseño original está pensado para relaciones multivariadas asincrónicas, no para el escenario univariado consolidado aquí.

8.2. Contraste con la hipótesis

La hipótesis general queda respaldada con matices. Sí es posible entrenar modelos basados en *Transformers* directamente sobre timestamps reales, sin imputar ni remuestrear la serie a una grilla temporal fija. También se verifica que los tokens objetivo permiten formular el pronóstico multi-horizonte como consultas futuras explícitas.

El matiz principal es arquitectónico. La formulación encoder pura de CT-Transformer es viable y competitiva, pero no fue el mejor sistema del benchmark final. El mejor desempeño aparece al separar historia y objetivos en una arquitectura encoder–decoder y luego ajustar autoregresivamente el decoder. En consecuencia, la contribución más fuerte de esta memoria no es sólo usar un encoding temporal continuo dentro de un encoder *Transformer*, sino mostrar que la representación con timestamps reales y tokens objetivo puede extenderse naturalmente a un esquema encoder–decoder autoregresivo con mejor desempeño.

La hipótesis específica de que el encoding temporal continuo sería siempre el componente crítico no queda confirmada por las ablaciones. En estos cinco datasets, la señal más clara proviene del identificador de token objetivo y del ajuste autoregresivo. El tiempo continuo sigue siendo conceptualmente relevante para evitar regularización a una grilla, pero su efecto empírico aislado requiere más evidencia.

8.3. Limitaciones

Las conclusiones anteriores deben leerse bajo cuatro limitaciones:

- La evaluación final usa cinco datasets. Es suficiente para cerrar esta fase y seleccionar un modelo ganador, pero no para sostener una afirmación universal.
- Las series consolidadas son univariadas. Esto limita especialmente la interpretación de CoFormer y deja pendiente la evaluación multivariada asincrónica.
- Las variantes autoregresivas se evaluaron con ocho objetivos contiguos, mientras que los modelos no autoregresivos usan hasta cuatro objetivos dentro del rango de offsets configurado. La comparación refleja el desempeño de los sistemas evaluados, pero no aísla completamente el efecto del protocolo de horizontes.
- No se realizó una búsqueda exhaustiva de hiperparámetros para cada arquitectura. Los resultados comparan configuraciones razonables y consistentes, no óptimos globales por familia.

8.4. Trabajo futuro

Las líneas de continuación más relevantes son:

1. **Ampliar la cobertura experimental:** repetir el análisis en más datasets y, si el costo lo permite, aumentar el número de semillas.
2. **Homogeneizar horizontes de evaluación:** comparar variantes autoregresivas y no autoregresivas sobre exactamente el mismo conjunto de timestamps objetivo.
3. **Extender a series multivariadas asincrónicas:** evaluar si CoFormer y las variantes con embeddings de sensor recuperan ventaja cuando existen múltiples variables observadas en tiempos distintos.
4. **Profundizar el ajuste autoregresivo:** estudiar si el beneficio proviene del decoder, de la máscara causal, del número de objetivos o del modo contiguo de entrenamiento.
5. **Explorar regularización y calibración:** medir no sólo error medio, sino estabilidad por horizonte, sesgo de predicción e incertidumbre.
6. **Optimizar costo computacional:** dado que los modelos pequeños rindieron mejor, conviene estudiar compresión, entrenamiento más corto y criterios de parada específicos para cada familia.

8.5. Cierre

La memoria permite concluir que los *Transformers* pueden adaptarse de forma efectiva a series de tiempo no equiespaciadas usando timestamps reales y tokens objetivo. El mejor modelo observado es un encoder-decoder pequeño con ajuste autoregresivo contiguo, que supera a CT-Transformer pequeño, CT-Transformer base, STraTS adaptado, persistencia y el modelo lineal temporal en los cinco datasets evaluados.

La conclusión final no es que una única pieza resuelva por completo el problema, sino que la combinación de representación temporal directa, tokens objetivo, separación encoder-decoder y generación autoregresiva constituye la ruta más prometedora encontrada en esta evaluación. Con más datasets y una comparación de horizontes completamente homogénea, esta línea tiene una base clara para convertirse en una validación más amplia.

Bibliografía

- Brockwell, P. J. (2001a). Continuous-time ARMA processes. *Handbook of statistics*, 19, 249-276.
- Brockwell, P. J. (2001b). Lévy-driven CARMA processes. *Annals of the Institute of Statistical Mathematics*, 53(1), 113-124.
- Che, Z., Purushotham, S., Cho, K., Sontag, D., & Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. *Scientific reports*, 8(1), 6085.
- Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. *Advances in neural information processing systems*, 31.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.
- De Brouwer, E., Simm, J., Arany, A., & Moreau, Y. (2019). GRU-ODE-Bayes: Continuous modeling of sporadically-observed time series. *Advances in neural information processing systems*, 32.
- Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10), 2451-2471.
- Hills, J., Lines, J., Baranauskas, E., Mapp, J., & Bagnall, A. (2014). Classification of time series by shapelet transformation. *Data mining and knowledge discovery*, 28(4), 851-881.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- Kazemi, S. M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., & Brubaker, M. (2019). Time2vec: Learning a vector representation of time. *arXiv preprint arXiv:1907.05321*.
- Kidger, P., Morrill, J., Foster, J., & Lyons, T. (2020). Neural controlled differential equations for irregular time series. *Advances in neural information processing systems*, 33, 6696-6707.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*.
- Liu, X., Qiu, X., Wu, X., Li, Z., Guo, C., Hu, J., & Yang, B. (2025). Rethinking Irregular Time Series Forecasting: A Simple yet Effective Baseline. *arXiv preprint arXiv:2505.11250*.
- Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. *International Conference on Learning Representations*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *International Conference on Learning Representations*.

- Moreno-Pino, F., Arroyo, Á., Waldon, H., Dong, X., & Cartea, Á. (2024). Rough transformers: Lightweight and continuous time series modelling through signature patching. *Advances in Neural Information Processing Systems*, 37, 106264-106294.
- Müller, M. (2007). *Information retrieval for music and motion*. Springer.
- Press, O., Smith, N. A., & Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Generalization. *International Conference on Learning Representations*.
- Sakoe, H., & Chiba, S. (2003). Dynamic programming algorithm optimization for spoken word recognition. *IEEE transactions on acoustics, speech, and signal processing*, 26(1), 43-49.
- Tipirneni, S., & Reddy, C. K. (2022). Self-Supervised Transformer for Sparse and Irregularly Sampled Multivariate Clinical Time-Series. *ACM Transactions on Knowledge Discovery from Data*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Wei, Y., Peng, J., He, T., Xu, C., Zhang, J., Pan, S., & Chen, S. (2023). Compatible Transformer for Irregularly Sampled Multivariate Time Series. *arXiv preprint arXiv:2310.11022*.
- Williams, C. K., & Rasmussen, C. E. (2006). *Gaussian processes for machine learning* (Vol. 2). MIT press Cambridge, MA.
- Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34, 22419-22430.
- Ye, L., & Keogh, E. (2009). Time series shapelets: a new primitive for data mining. *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 947-956.
- Yulia, R., Chen Ricky, T., & David, D. (2019). Latent ODEs for irregularly-sampled time series. *Advances in Neural Information Processing Systems*, 32.
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of the AAAI conference on artificial intelligence*, 35(12), 11106-11115.
- Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., & Jin, R. (2022). Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *International conference on machine learning*, 27268-27286.

Apéndice A

Fundamentos de redes neuronales y optimización

A.1. Introducción

El objetivo de este anexo es presentar de manera autocontenida los conceptos básicos necesarios para comprender el funcionamiento de los modelos de redes neuronales utilizados en esta memoria, incluyendo:

- El modelo de *neurona artificial* y su interpretación geométrica.
- La construcción de redes neuronales feedforward como composiciones de capas.
- El proceso de entrenamiento basado en funciones de pérdida, descenso de gradiente y retropropagación (*backpropagation*).
- El papel de las funciones de activación y sus propiedades.
- El algoritmo de optimización Adam, utilizado en los experimentos de esta memoria.

Aunque el cuerpo principal del documento se centra en arquitecturas específicas como los Transformers para series de tiempo, todos estos modelos se apoyan en los mismos bloques básicos: capas lineales, funciones de activación, composición de funciones diferenciables y algoritmos de optimización sobre grandes espacios de parámetros.

A.2. Neurona artificial

A.2.1. Modelo básico

La unidad fundamental en una red neuronal es la **neurona artificial** o **perceptrón**, que puede verse como un modelo matemático que recibe un vector de entrada, aplica una transformación lineal y luego una función no lineal.

Sea $x \in \mathbb{R}^d$ un vector de entrada (por ejemplo, mediciones de sensores en un instante de tiempo), $w \in \mathbb{R}^d$ un vector de pesos y $b \in \mathbb{R}$ un término de sesgo (*bias*). El modelo de neurona se escribe como:

$$z = w^\top x + b, \tag{A.1}$$

$$y = \phi(z), \quad (\text{A.2})$$

donde:

- z es la combinación lineal de la entrada.
- ϕ es la **función de activación**, que introduce no linealidad.
- y es la salida de la neurona.

La presencia de ϕ es indispensable porque la composición de transformaciones lineales sigue siendo lineal, sin una activación, la neurona quedaría limitada a representar relaciones objetivo de ese mismo tipo.

A.2.2. Interpretación geométrica

Desde un punto de vista geométrico, la expresión $w^\top x + b = 0$ describe un hiperplano en $\mathbb{R}^d$. La neurona aprende un vector w y un sesgo b que determinan este hiperplano, y la función de activación ϕ determina cómo se asignan valores de salida a cada lado de dicho hiperplano.

Por ejemplo, si ϕ es una función escalón (no diferenciable, pero útil como intuición), la neurona implementa un clasificador lineal que decide a qué lado del hiperplano pertenece el punto x .

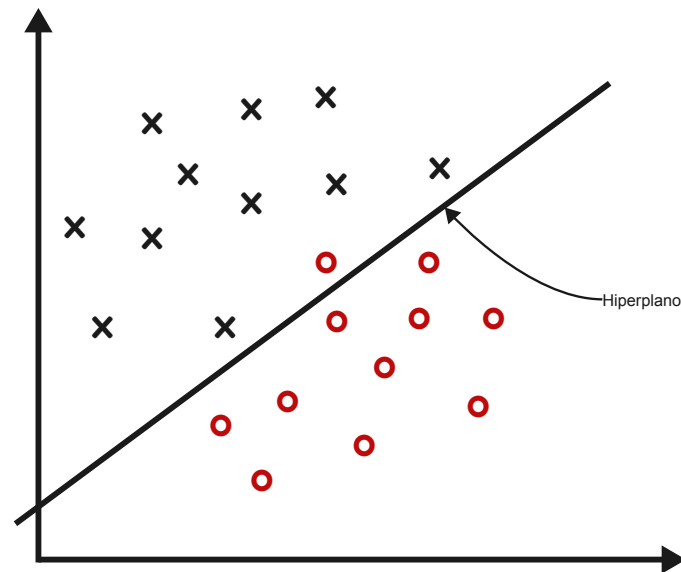


Figura A.1: Ejemplo bidimensional de un hiperplano que separa dos clases linealmente separables: las cruces negras pertenecen a una clase y los círculos rojos a la otra.

A.3. Redes neuronales feedforward

A.3.1. Capas densas y notación matricial

Una **capa densa** (o *fully-connected*) agrupa muchas neuronas en paralelo. Sea $x \in \mathbb{R}^{d_{\text{in}}}$ la entrada a la capa. Una capa densa con d_{out} neuronas se define como:

$$z = Wx + b, \quad z \in \mathbb{R}^{d_{\text{out}}}, \quad (\text{A.3})$$

$$y = \phi(z), \quad (\text{A.4})$$

donde:

- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ es la matriz de pesos.
- $b \in \mathbb{R}^{d_{\text{out}}}$ es el vector de sesgo.
- ϕ se aplica componente a componente. ($\phi : \mathbb{R}^{d_{\text{out}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$, donde $\phi(z) = (\phi(z_i))_i$)

Para un conjunto de N ejemplos, se suele apilar las entradas en una matriz $X \in \mathbb{R}^{N \times d_{\text{in}}}$ y la salida en $Y \in \mathbb{R}^{N \times d_{\text{out}}}$.

A.3.2. Composición de capas

Una **red neuronal feedforward** (o *Multilayer Perceptron*, MLP) consiste en la composición de varias capas de este tipo. Por ejemplo, con dos capas ocultas:

$$h_1 = \phi_1(W_1x + b_1), \quad (\text{A.5})$$

$$h_2 = \phi_2(W_2h_1 + b_2), \quad (\text{A.6})$$

$$\hat{y} = W_3h_2 + b_3, \quad \hat{y} \in \mathbb{R}^{d_{\text{out}}}, \quad (\text{A.7})$$

donde:

- x es la entrada.
- h_1, h_2 son representaciones internas (*features* aprendidas).
- $\hat{y}$ es la salida del modelo (por ejemplo, una predicción de la serie de tiempo).

Este esquema se generaliza a un número arbitrario de capas. La capacidad de aproximación universal de las redes neuronales feedforward (bajo condiciones suaves sobre las funciones de activación) asegura que, con suficiente ancho y profundidad, pueden aproximar funciones muy complejas.

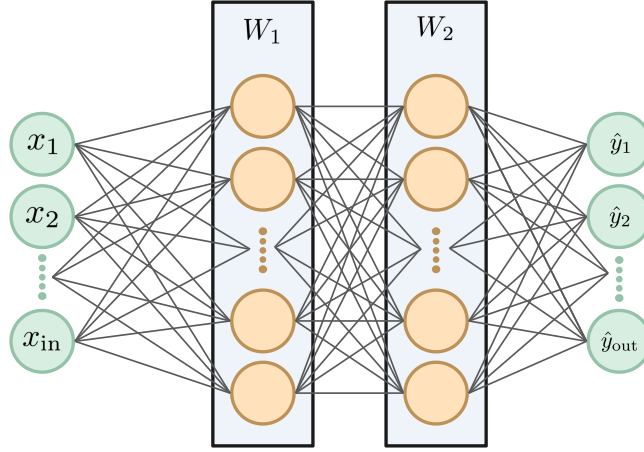


Figura A.2: Esquema de una red neuronal *feedforward* con dos capas ocultas: los vectores de entrada $x_1, \dots, x_{in}$ se transforman mediante las matrices de pesos W_1 y W_2 en representaciones internas, para finalmente producir las salidas $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{out}$.

A.4. Función de pérdida

A.4.1. Pérdida para problemas de regresión

En esta memoria el problema principal es de **regresión**: se quiere predecir un valor continuo (por ejemplo, el valor de una serie de tiempo en un instante futuro). Sea $\hat{y}_i$ la predicción del modelo para el ejemplo i y y_i el valor real. La función de pérdida más habitual es el **error cuadrático medio** (*Mean Squared Error*, MSE):

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2, \quad (\text{A.8})$$

donde θ representa el conjunto de todos los parámetros del modelo (todas las matrices W y vectores b).

El objetivo del entrenamiento es encontrar θ que minimice esta función de pérdida sobre el conjunto de datos de entrenamiento.

Otras pérdidas útiles son, por ejemplo:

- **Error absoluto medio** (MAE):

$$\mathcal{L}_{\text{MAE}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_1.$$

- **Pérdida de Huber**, que combina propiedades de MSE y MAE, siendo más robusta a outliers.

A.4.2. Regularización en la función de pérdida

Para evitar sobreajuste (*overfitting*), a menudo se agrega un término de **regularización** a la pérdida. Por ejemplo, regularización L_2 (también conocida como *weight decay*):

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{datos}}(\theta) + \lambda \sum_j \|W_j\|_F^2, \quad (\text{A.9})$$

donde $\lambda > 0$ controla la fuerza de la regularización, y la suma recorre las matrices de pesos W_j de cada capa.

A.5. Descenso de gradiente y retropropagación

A.5.1. Descenso de gradiente

El procedimiento básico para ajustar los parámetros de la red consiste en aplicar **descenso de gradiente**. Dada una función de pérdida $\mathcal{L}(\theta)$, el descenso de gradiente estándar actualiza los parámetros según:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad (\text{A.10})$$

donde:

- θ_t es el vector de parámetros en el paso t .
- $\eta > 0$ es la tasa de aprendizaje (*learning rate*).
- $\nabla_{\theta} \mathcal{L}(\theta_t)$ es el gradiente de la pérdida respecto de los parámetros.

En la práctica, los datasets suelen ser grandes, por lo que calcular el gradiente sobre todos los datos en cada paso es costoso. Por ello se utiliza **descenso de gradiente estocástico** (SGD) o variantes *mini-batch*, en las que el gradiente se estima a partir de un subconjunto de ejemplos:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}(\theta_t), \quad (\text{A.11})$$

donde $\mathcal{B}$ es un mini-batch de índices de entrenamiento.

A.5.2. Retropropagación (*backpropagation*)

Para poder aplicar descenso de gradiente, se requiere calcular el gradiente de la pérdida respecto de todos los parámetros de la red. Esto se logra mediante el algoritmo de **retropropagación**, que se basa en la regla de la cadena del cálculo diferencial.

A nivel conceptual, la red se puede ver como una composición de funciones:

$$\hat{y} = f_L \circ f_{L-1} \circ \cdots \circ f_1(x; \theta), \quad (\text{A.12})$$

donde cada f_{ℓ} representa una capa y depende de un subconjunto de parámetros θ_{ℓ} .

La regla de la cadena indica que:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\ell}} = \frac{\partial \mathcal{L}}{\partial f_{\ell}} \cdot \frac{\partial f_{\ell}}{\partial \theta_{\ell}}, \quad (\text{A.13})$$

y que las derivadas $\frac{\partial \mathcal{L}}{\partial f_{\ell}}$ se pueden calcular recursivamente desde la salida hacia la entrada.

En términos prácticos:

1. **Paso forward:** se propaga la entrada a través de la red y se calcula la pérdida.
2. **Paso backward:** se calcula el gradiente de la pérdida respecto de la salida y se va propagando hacia atrás, capa por capa, acumulando las derivadas parciales.
3. **Actualización:** se actualizan los parámetros usando el gradiente calculado y el algoritmo de optimización elegido (SGD, Adam, etc.).

En frameworks modernos como PyTorch, este proceso se implementa automáticamente mediante *autograd*, siempre que las operaciones definidas en el modelo sean diferenciables.

A.6. Funciones de activación

A.6.1. Requisitos básicos

La función de activación ϕ cumple dos roles fundamentales:

- Introducir **no linealidad**. Sin funciones de activación, la composición de capas lineales sigue siendo una transformación lineal global.
- Ser **diferenciable** (al menos por tramos) para poder aplicar retropropagación.

A continuación se describen algunas de las funciones de activación más utilizadas.

A.6.2. Sigmoide y tangente hiperbólica

La función **sigmoide** se define como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (\text{A.14})$$

Sus valores están en el intervalo $(0, 1)$ y su derivada es:

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)). \quad (\text{A.15})$$

La **tangente hiperbólica** se define como:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad (\text{A.16})$$

con valores en $(-1, 1)$ y derivada:

$$\tanh'(z) = 1 - \tanh^2(z). \quad (\text{A.17})$$

Ambas funciones se usaron ampliamente en redes neuronales clásicas, pero presentan el problema de **gradientes que se desvanecen** (*vanishing gradients*) para valores de $|z|$ grandes, lo que dificulta el entrenamiento de redes profundas.

A.6.3. ReLU y variantes

La **Rectified Linear Unit** (ReLU) se define como:

$$\text{ReLU}(z) = \max(0, z). \quad (\text{A.18})$$

Su derivada es:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Las ReLU han sido muy exitosas en redes profundas porque mitigan parcialmente el problema de gradientes que se desvanecen en la región $z > 0$ y son computacionalmente simples. Sin embargo, pueden producir neuronas “muertas” (salida cero constante) si los pesos se actualizan de forma que z permanezca en la región negativa.

Variantes como **Leaky ReLU** introducen un pequeño gradiente en la región negativa:

$$\text{LeakyReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases}$$

con α pequeño (por ejemplo, $\alpha = 0,01$).

En modelos más recientes, incluyendo Transformers, es frecuente usar activaciones más suaves como **GELU** (*Gaussian Error Linear Unit*), que puede verse como una versión suavizada de ReLU con ciertas propiedades teóricas favorables.

A.7. Problemas en el entrenamiento de redes profundas

A.7.1. Gradientes que se desvanecen y explotan

En redes profundas, al aplicar retropropagación, se multiplican muchas matrices y derivadas de funciones de activación. Esto puede llevar a que:

- Los gradientes se hagan extremadamente pequeños (**vanishing gradients**), dificultando el aprendizaje en capas tempranas.
- Los gradientes crezcan de forma descontrolada (**exploding gradients**), causando inestabilidad numérica.

Existen varias estrategias para mitigar estos problemas:

- Diseñar buenas **inicializaciones** de los pesos (por ejemplo, He o Xavier).
- Utilizar funciones de activación adecuadas (ReLU, GELU, etc.).
- Aplicar **normalizaciones**, como *Batch Normalization* o *Layer Normalization*.
- Aplicar **clipping de gradiente**, limitando la norma del gradiente en cada actualización.

A.7.2. Regularización y generalización

Además de los términos de regularización en la pérdida, existen técnicas adicionales para mejorar la capacidad de generalización:

- **Dropout:** durante el entrenamiento, se anulan aleatoriamente algunas neuronas (se ponen a cero) con cierta probabilidad, lo que fuerza al modelo a no depender excesivamente de un subconjunto pequeño de unidades.
- **Early stopping:** se detiene el entrenamiento cuando la pérdida de validación deja de mejorar, evitando sobreajuste.
- **Data augmentation:** en dominios como imágenes o texto, se generan ejemplos adicionales modificando ligeramente los datos originales. En series de tiempo esto es más delicado, pero existen técnicas específicas.

A.8. Optimizadores avanzados: Adam

A.8.1. Motivación

El descenso de gradiente estándar utiliza una única tasa de aprendizaje η para todos los parámetros y no tiene memoria explícita del historial de gradientes. En problemas complejos y de alta dimensión, esto puede hacer que:

- El entrenamiento sea sensible a la elección de η .
- El modelo tarde en converger o quede atrapado en regiones poco favorables.

Por ello se han desarrollado **optimizadores avanzados** que adaptan la tasa de aprendizaje de manera automática, utilizando información adicional sobre la historia del gradiente. Uno de los más usados en la práctica es **Adam** (*Adaptive Moment Estimation*).

A.8.2. Algoritmo Adam

Sea θ_t el vector de parámetros en el paso t y $g_t = \nabla_{\theta} \mathcal{L}_t(\theta_t)$ el gradiente de la pérdida calculado en ese paso (por ejemplo, sobre un mini-batch). Adam mantiene dos variables auxiliares:

- m_t : estimación del primer momento (media) del gradiente.
- v_t : estimación del segundo momento (varianza no centrada) del gradiente.

El algoritmo puede describirse como sigue:

1. Inicializar $m_0 = 0$, $v_0 = 0$, $t = 0$.

2. En cada paso:

$$t \leftarrow t + 1, \tag{A.19}$$

$$g_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_t), \tag{A.20}$$

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{A.21}$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{A.22}$$

donde las operaciones son componente a componente y $0 < \beta_1, \beta_2 < 1$ son hiperparámetros (típicamente $\beta_1 \approx 0,9$, $\beta_2 \approx 0,999$).

3. Para corregir el sesgo inicial (ya que m_t y v_t parten en cero), se calculan:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (\text{A.23})$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (\text{A.24})$$

4. Finalmente, se actualizan los parámetros:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad (\text{A.25})$$

donde η es la tasa de aprendizaje base y ε es un pequeño valor positivo (por ejemplo, 10^{-8}) para evitar divisiones por cero.

En palabras simples:

- m_t actúa como un **promedio móvil** del gradiente (similar a *momentum*).
- v_t actúa como un promedio móvil de la magnitud cuadrática del gradiente, permitiendo **adaptar** la escala de los pasos por parámetro.
- La división por $\sqrt{\hat{v}_t}$ hace que los parámetros con gradientes consistentemente grandes tengan pasos de actualización más pequeños, y viceversa.

En esta memoria se utiliza AdamW como optimizador principal para entrenar los modelos Transformer, tal como se especifica en los archivos de configuración de entrenamiento. AdamW conserva el carácter adaptativo de Adam, pero desacopla el decaimiento de pesos de la actualización basada en momentos.

A.9. Relación con CT-Transformer

Aunque el Transformer parece, a primera vista, una arquitectura muy distinta a un MLP estándar, en el fondo reutiliza los mismos componentes básicos descritos en este anexo:

- Cada bloque de **atención multi-cabeza** está compuesto por proyecciones lineales (capas densas) que calculan las matrices de *queries*, *keys* y *values*, seguidas de operaciones de atención y una proyección lineal de salida.
- El **bloque feed-forward** dentro de cada capa del Transformer es esencialmente un MLP de dos capas aplicado posición a posición, con una función de activación no lineal (por ejemplo, ReLU o GELU).
- La red completa se entrena minimizando una función de pérdida (en este caso, típicamente MSE sobre las predicciones de la serie de tiempo) usando descenso de gradiente y un optimizador como Adam o AdamW.
- La **cabeza de regresión** sobre el token *target* en el modelo de esta memoria es, nuevamente, una capa lineal (o una pequeña MLP) que transforma la representación latente del token en la predicción final.

Por lo tanto, comprender el funcionamiento de neuronas, funciones de activación, retropropagación y optimizadores adaptativos como Adam o AdamW es suficiente para entender el entrenamiento de toda la arquitectura CT-Transformer, aunque los detalles de la atención, los encodings temporales y el scheduler usado en los experimentos se desarrollen en capítulos específicos y en el Anexo B.

A.10. Resumen del anexo

En este anexo se han presentado los fundamentos necesarios para comprender el entrenamiento de redes neuronales modernas:

- El modelo de neurona artificial como combinación lineal más función de activación.
- La construcción de redes feedforward mediante la composición de capas densas.
- La definición de funciones de pérdida para problemas de regresión y el uso de regularización.
- El algoritmo de entrenamiento basado en descenso de gradiente, estimado mediante mini-batches, y el cálculo eficiente de gradientes mediante retropropagación.
- El rol de las funciones de activación y los problemas asociados al entrenamiento de redes profundas (gradientes que se desvanecen o explotan).
- El algoritmo Adam y su variante AdamW como optimizadores adaptativos ampliamente utilizados en la práctica.

Estos conceptos constituyen la base sobre la cual se construyen las arquitecturas específicas analizadas en la memoria, en particular CT-Transformer.

Apéndice B

Detalles de optimización y scheduler

Este anexo complementa la descripción general de optimización presentada en el Anexo A. En particular, se detalla la configuración usada en el benchmark final: optimización con AdamW y una política de tasa de aprendizaje con calentamiento lineal seguido de decaimiento coseno.

B.1. AdamW

Adam estima momentos de primer y segundo orden del gradiente para adaptar la escala de actualización de cada parámetro Kingma y Ba, 2015. En la práctica, para modelos Transformer se usa frecuentemente AdamW, que separa la regularización por decaimiento de pesos de la actualización adaptativa Loshchilov y Hutter, 2019. Esta separación evita mezclar el término de regularización con los momentos acumulados del gradiente.

Si θ_t representa los parámetros, η_t la tasa de aprendizaje de la época o paso actual, y m_t , v_t los momentos corregidos por sesgo, AdamW puede verse esquemáticamente como:

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_t \right), \quad (\text{B.1})$$

donde λ controla el decaimiento de pesos. La idea importante para esta memoria es que η_t no se mantiene constante: se modifica durante el entrenamiento mediante un scheduler.

B.2. Calentamiento lineal y decaimiento coseno

En los experimentos principales no se usa un CosineAnnealingLR puro, sino una variante con **calentamiento lineal** (*linear warmup*) seguido de **decaimiento coseno**. El calentamiento lineal evita comenzar el entrenamiento con una tasa de aprendizaje demasiado alta cuando los pesos todavía no han estabilizado sus representaciones internas. Esta práctica es especialmente útil en arquitecturas Transformer, donde las capas de atención, normalización y proyecciones lineales pueden ser sensibles a los primeros pasos de optimización.

Sea $\eta_{\text{máx}}$ la tasa de aprendizaje base, E el número máximo de épocas, w el número de épocas de calentamiento y e la época actual, indexada desde 0. La tasa de aprendizaje efectiva se define como:

$$\eta(e) = \eta_{\text{máx}} \cdot s(e), \quad (\text{B.2})$$

donde

$$s(e) = \begin{cases} \frac{e+1}{w}, & e < w, \\ \frac{1}{2} \left[1 + \cos \left(\pi \frac{e-w}{E-w} \right) \right], & e \geq w. \end{cases} \quad (\text{B.3})$$

La primera rama aumenta la tasa de aprendizaje de forma aproximadamente lineal hasta $\eta_{\text{máx}}$. La segunda rama aplica un decaimiento suave con forma coseno, relacionado con la familia de políticas de *cosine annealing* Loshchilov y Hutter, 2017, hasta acercarse a cero al final del entrenamiento.

En la implementación, el scheduler se actualiza una vez por época. Para modelos pequeños y medianos se utiliza $E = 30$ con $w = 2$, para modelos grandes se utiliza $E = 60$ con $w = 5$. En variantes más inestables se mantiene la misma forma funcional, pero se aumenta el calentamiento mínimo a cinco épocas y se reduce la tasa de aprendizaje base.

B.3. Diferencia con CosineAnnealingLR

CosineAnnealingLR corresponde al decaimiento coseno sin etapa inicial de calentamiento. En cambio, la configuración principal de esta memoria usa una política compuesta:

$$\text{linear warmup} \longrightarrow \text{cosine decay}.$$

Por lo tanto, cuando en el texto se habla de decaimiento coseno debe entenderse esta versión con calentamiento, salvo que se indique explícitamente lo contrario. Esta precisión es relevante porque el calentamiento modifica las primeras épocas de entrenamiento y fue parte de la receta usada para mejorar estabilidad en el benchmark final.